esprit

Se former autrement

2025 – 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY: INFORMATION TECHNOLOGY

Design of a Multi-Agent AI Pipeline for Evidence Collection from Illegal Streaming Sites

By: Ahmed ARFAOUI

Academic supervisor: [Academic Supervisor]

Corporate Internship Supervisor: Wael Hkiri

LOGO ENTREPRISE

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- **Nom & Prénom / Name & Surname :** Ahmed ARFAOUI

Encadrant Entreprise / Business site Supervisor

- **Nom & Prénom / Name & Surname :** Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique / Academic Supervisor

- **Nom & Prénom / Name & Surname :** [Academic Supervisor]

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné** / This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Abstract

This report presents the design, development, and evaluation of an agentic evidence-collection system for illegal streaming websites. The work was carried out during an internship at Soft Stars for the client beIN SPORTS. The objective was to support rights-enforcement workflows by visiting target websites, classifying their pages, extracting hosting and stream evidence, and preserving auditable outputs such as screenshots, traces, and structured results.

The internship produced two related artifacts: a validated **n8n** workflow, from which one operational agent was deployed in the company context, and a **Python**-based reference architecture designed to improve maintainability, observability, memory, evaluation, and future scalability. The company-side operational outcome therefore remained a focused single-agent workflow, while the broader FastAPI/Puppeteer multi-agent implementation is presented as an academic and engineering reference architecture derived from the validated n8n workflow.

The proposed architecture organizes evidence collection around specialized reasoning agents, browser automation tools, structured outputs, trace storage, screenshot capture, and evaluation metrics. Its target deployment model separates API services, browser workers, observability storage, screenshot storage, and queue-based execution for future Azure Container Apps Jobs operation. The Python architecture does not depend on the enterprise production database; the database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs.

Acknowledgements

I would like to thank my academic and professional supervisors for their guidance throughout this internship. I also thank the Soft Stars team for the technical discussions, feedback, and trust that made this work possible.

Contents

CHAPTER 1

Introduction

1.1 Host organization — Soft Stars

Soft Stars is a software company specializing in digital media intelligence and content protection solutions. The company develops technical platforms for monitoring, detecting, and documenting unauthorized redistribution of premium audiovisual content across global streaming piracy networks. Its engineering work spans automated browser workflows, web intelligence, data enrichment pipelines, and rights-enforcement tooling built to the operational requirements of major media rights holders.

This internship was conducted within Soft Stars for the client **beIN SPORTS**, one of the world's largest sports broadcasting networks. The official project title was *Design of a Multi-Agent AI Pipeline for Evidence Collection from Illegal Streaming Sites*. The mission was to design, build, and evaluate a platform capable of visiting suspicious target websites, understanding their page structure, extracting stream evidence, and producing structured, auditable outputs that support legal and operational enforcement action.

1.2 Sports piracy and the evidence collection challenge

The broadcasting rights market for premium football and sports content represents an industry worth tens of billions of dollars globally. Rights holders such as beIN SPORTS invest heavily in acquiring exclusive licences that entitle them to distribute specific events to specific territories. Illegal streaming sites undermine those rights by redistributing protected content without authorisation, often at no cost to the viewer. The scale of the problem is significant: hundreds of aggregator domains may broadcast the same protected

match simultaneously during a major competition window [25, 26].

From an enforcement perspective, the first obstacle is *evidence*. Before a rights holder can act legally or operationally, it needs to prove:

- that a specific URL was accessible during a specific broadcast window;
- that the page delivered a live or on-demand stream of protected content;
- that the stream was hosted or relayed through identifiable infrastructure;
- that the stream resolved to a playable URL (typically an HLS `.m3u8` manifest or MPEG-DASH `.mpd` file).

Producing that proof at scale is an engineering problem. Manual monitoring cannot cover hundreds of domains simultaneously. A static HTML scraper cannot handle pages that require JavaScript execution, user interaction, or network-level observation to reach the stream URL. A rule-based system tied to fixed CSS selectors breaks every time a site operator updates their template. The internship project addresses this gap: how to collect the necessary evidence automatically, at scale, and traceably across a heterogeneous and continuously changing population of streaming sites.

1.3 Why the problem requires an adaptive approach

Three structural properties of illegal streaming sites make them resistant to conventional automation.

Structural heterogeneity. The target population includes sites built on dozens of different technology stacks: WordPress-derived templates shared across Arabic-language site families, custom React or Vue frontends on global hubs, PHP-driven backends with client-side overlay logic, and fully bespoke single-page applications. No single selector or fixed crawl path works across all of them.

Deliberate obfuscation. Site operators are incentivised to make automation difficult. Common countermeasures include Cloudflare bot-challenge pages, JavaScript-injected play buttons that do not exist in the initial HTML, advertising overlays that must be dismissed before the player is accessible, tokenized stream URLs that expire within seconds and require player activation before the manifest is generated, and nested iframe trees that distribute the player across two or three origin domains.

Continuous change. Domains rotate frequently. Layouts are updated between broadcast windows. Anti-bot configurations are adjusted in response to detection. Effective

evidence collection therefore requires not just a working solution but a *maintainable* one where the adaptation cost per site family is minimized.

These three properties together motivate the use of AI agents. An agent equipped with browser tools, a structured reasoning prompt, and persistent memory of past observations can adapt its extraction strategy at runtime in response to what it actually finds on the page.

1.4 Project evolution: from validated workflow to reference architecture

The internship did not begin with a fully specified reference architecture. The first phase used **n8n** as the main experimentation environment because it allowed rapid assembly of browser actions, early prompt testing on real targets, and end-to-end workflow validation with minimal development overhead. This phase established that agent-guided evidence collection was feasible on representative streaming sites and, after validation, led to the deployment of a focused **single-agent n8n workflow** in the company context.

As the scope matured, the limits of the visual workflow became clearer: prompt evolution was harder to version, browser traces were harder to structure, cross-run memory was limited, and systematic evaluation required more explicit engineering surfaces. This motivated a second phase built with **FastAPI**, **Next.js**, **Puppeteer**, structured persistence, and evaluation tooling. The Python implementation should therefore be understood as a **reference architecture** and a future engineering evolution derived from the validated n8n workflow, not as a claim that the enterprise deployed the full multi-agent platform in production.

The value of this second phase lies in making the artifact easier to maintain, observe, extend, and evaluate. It formalizes the modular agent boundaries, browser-tool contracts, trace model, memory model, and cloud-oriented execution target that would be difficult to express cleanly in n8n alone.

1.5 Problem statement

The main problem addressed by this internship can be stated as:

> How can we design a maintainable and extensible agent-based evidence-collection architecture that analyses illegal streaming websites, distinguishes different page roles, extracts evidence-grade stream and infrastructure

data, and remains effective despite dynamic interfaces, multilingual layouts, anti-bot barriers, and continuously evolving site behavior?

This problem is difficult for several reasons:

- a single website can expose landing pages, hosting pages, embedded players, and auxiliary pages in the same session;
- the evidence (stream URLs, screenshots, iframe chains) may appear only after clicks, navigation, or player activation;
- page structure changes frequently, making brittle hard-coded rules obsolete quickly;
- LLM inference cost grows fast if agent context and tool allocation are not controlled carefully.

1.6 Objectives and contributions

The main objective was to design and evaluate an agentic evidence-collection pipeline capable of processing suspicious streaming websites end to end. In practice, this objective was addressed through a validated n8n workflow and a Python reference architecture. This breaks down into the following operational goals:

- classify each input URL before deeper extraction begins;
- route the URL toward specialized agents according to page type;
- collect hosting links, embedded links, and stream URLs through browser-assisted adaptive extraction;
- preserve screenshots, tool-call traces, and final results for auditable review;
- analyze the infrastructure behind extracted streams;
- support takedown preparation with structured evidence output;
- expose the system through an operational interface and a programmatic API;
- prepare the solution for containerized event-driven deployment on Azure.

The internship produced contributions at both operational-workflow and reference-architecture level:

- **Validated n8n workflow and company-side deployment**: rapid prototype used to validate the extraction pipeline, from which one focused single-agent workflow was deployed in the company context;
- **Prompt and browser-interaction experimentation**: iterative refinement of page classification, tab navigation, player activation, popup handling, and iframe traversal on real target websites;
- **Validation metrics and traces**: collection of screenshots, tool traces, success/failure observations, and token/cost measurements from representative

runs;

- **Python/FastAPI reference architecture**: academic implementation derived from the validated workflow, emphasizing maintainability, modular orchestration, observability, memory, evaluation, and future scalability;
- **Puppeteer tool layer**: six browser tools (inspect, interact, harvest, screenshot, navigate, get_media_state) enabling adaptive page interaction beyond static HTML retrieval;
- **Observability and evaluation database**: persistence for runs, events, tool calls, errors, screenshots, metrics, and structured outputs, separate from any enterprise production database;
- **Operator console concept/prototype**: FastAPI backend and Next.js review surface for live monitoring, history inspection, and evaluation review;
- **Future deployment target**: queue-driven architecture using Azure Service Bus and Azure Container Apps Jobs as the intended execution model for later industrialization.

1.7 Report structure

Chapter 2 introduces the technical background and key technologies. Chapter 3 presents the adopted methodology, derives requirements, and documents design constraints. Chapter 4 describes the system architecture and target Azure deployment model. Chapter 5 explains the implementation in detail: agent composition, prompt engineering, and browser tooling. Chapter 6 evaluates the platform through automated tests, manual campaigns, and prompt evolution metrics. Chapter 7 concludes and outlines future directions.

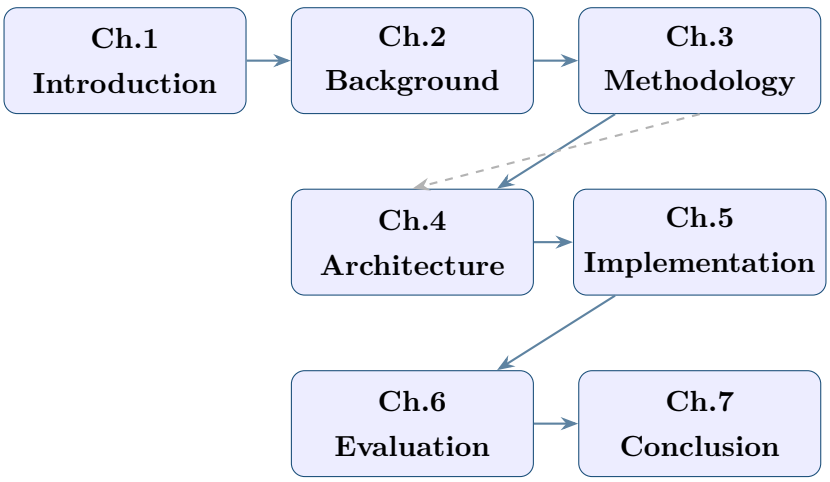

Figure 1.1: Report reading map: chapter sequence and main dependencies.

CHAPTER 2

Technical Background and Working Tools

This chapter establishes the technical context required to understand the design choices made in subsequent chapters. It covers the structural challenges of illegal streaming websites, the core technologies selected to address them, the streaming protocols that the platform is designed to capture, and the residual limitations that motivate an adaptive agentic approach rather than a fixed scraper.

2.1 Why this domain is hard in practice

Illegal streaming websites are difficult to automate because they are built for volatility. In a single run, the same domain can present list pages, server switchers, nested player frames, popups, redirects, and anti-bot interstitials. A system that succeeds only on static HTML snapshots will fail frequently in this environment.

From repeated runs, the main difficulties were:

- content that appears only after user-like interaction;
- deeply nested iframe trees where the useful player is not in the top frame;
- unstable selectors caused by obfuscation and frequent layout changes;
- anti-bot behaviors that escalate after repetitive scripted actions;
- stream URLs that are signed, short-lived, or generated after runtime JavaScript logic.

2.2 From deterministic scripts to adaptive agents

The core difficulty in this domain is that no fixed rule generalizes across all page structures. A deterministic scraper works well on stable pages with known selectors, but fails when controls appear only after interaction, when the useful player is buried in an iframe chain, or when the site changes behavior between sessions. For this reason, the project moved toward an adaptive browser-assisted approach in which the system observes the current page state, chooses the next action, and re-evaluates the result before proceeding. Research on agentic reasoning and realistic web benchmarks such as WebArena supports this kind of iterative decision-making on dynamic interfaces [3, 8].

2.3 ReAct as the operating pattern

In practice, the specialist agents follow a ReAct-style loop [1]: observe the current browser state, reason about the next step, act through a browser tool, and check whether the action produced useful evidence. This operating pattern is better suited to streaming sites than a linear workflow because it can recover from false clicks, inactive server buttons, popup interruptions, and empty first attempts.

The practical advantages for this project are straightforward:

- the next action depends on the actual page state, not on a fixed branch decided in advance;
- every important step can be validated through DOM, screenshot, or network evidence;
- the run can stop early when evidence is found, or fall back when the current path fails.

2.4 Observed website families in practice

During manual and semi-automated campaigns, a broad range of streaming-site structures were encountered. They can be organized into two main axes: *simple/static sites* (those with predictable HTML and stable selectors) and *complex/interactive sites* (those requiring JavaScript navigation and dynamic interaction to reveal content).

2.4.1 Simple sites: WordPress-template regional aggregators

A large percentage of regional Arabic-language football streaming portals are built on the same WordPress theme family. Several sites in this category were observed to share identical CSS class names, the same component hierarchy, and even the same player plugin (AlbaPlayer or Clappr embedded via a shortcode). This made them the easiest target family: once a selector pattern was confirmed on one site, it often transferred directly to another within the same template cluster.

Operational traits:

- match pages from repeating templates where visible layout is stable;
- server buttons with predictable class names (`.server-btn`, `.stream-link`) that can be targeted by selector;
- ad and redirect logic injected by plugins or external `<script>` tags;
- occasional template refresh that preserves URL structure but updates class names.

These sites are called "simple" because a well-formed selector-based interact call succeeds on the first attempt in the majority of runs.

2.4.2 Complex sites: JavaScript-heavy interactive portals

Complex sites require navigational interaction before useful content becomes visible. Common patterns include:

- **Tab-gated match listings:** the landing page shows only live matches by default; other events are behind tab controls that trigger JavaScript fetches (no page reload) when clicked;
- **Dynamic server loading:** server options are not in the initial HTML but are injected by an XHR response triggered by selecting a server tab;
- **Scroll-gated content:** match cards below the fold are lazy-loaded and only appear after a scroll event reaches a threshold;
- **Login-wall or subscription interstitials:** some mirrors show a blurred player until a fake login form is dismissed.

These sites require the landing or hosting agent to perform a multi-step navigation loop (inspect $\rightarrow$ interact $\rightarrow$ re-inspect) before extraction can begin.

2.4.3 Global link-hub aggregators

Global indexes (such as streamed-style hubs) frequently act as routing layers rather than final player pages. They are neither purely Arabic nor purely WordPress; their structure is custom but their routing behavior is consistent:

- large event listings with many outbound host domains;
- per-event pages that aggregate multiple mirrors with inconsistent UI patterns;
- rapid provider rotation where one mirror fails while another becomes active;
- mixed language/region metadata that complicates deterministic field extraction.

Site type	Extraction difficulty	Selector stability	Agent needed
WordPress regional (Arabic)	Low–Medium	High (shared CSS classes)	Hosting agent with basic interact
JS-interactive portals	High	Low (dynamic injection)	Landing + hosting with nav loops
Global link-hub	Medium–High	Medium (custom but repeating)	Full pipeline with embedded fallback

Table 2.1: Site-family classification by extraction difficulty and agent strategy.

2.5 Three-layer page taxonomy

Manual analysis of many streaming websites revealed a consistent three-layer structure that the pipeline exploits. Figure 2.1 shows how a single investigation progresses through these layers.

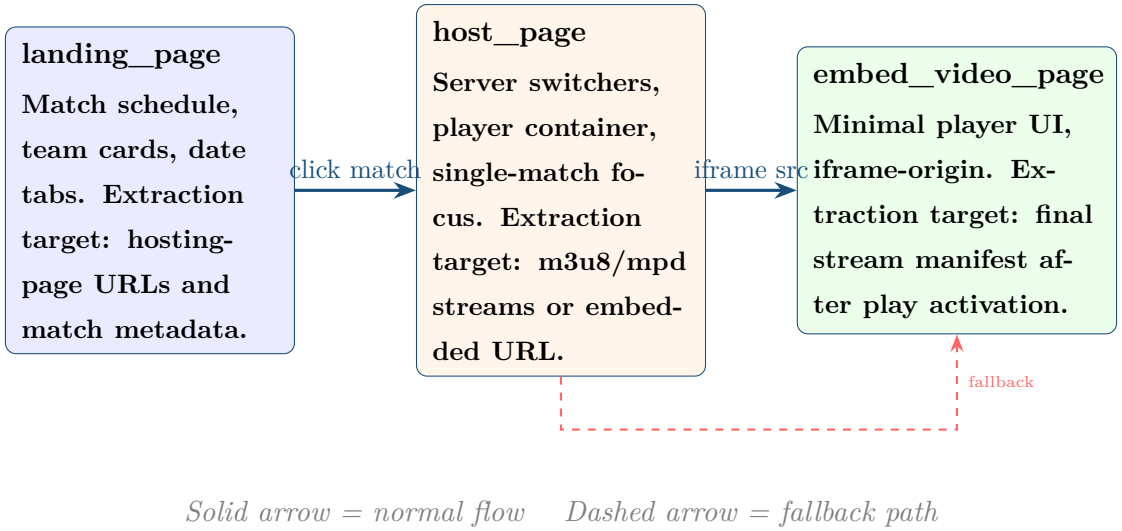

Figure 2.1: Three-layer taxonomy of streaming-site page types and the extraction flow.

2.6 Why returning only HTML does not work

For this project, plain HTML retrieval is insufficient for five concrete reasons:

1. many pages render critical controls through JavaScript after load;
2. active stream URLs often appear only in network requests after play/click actions;
3. iframe content can be isolated and absent from the top-level DOM snapshot;
4. anti-bot defenses may show alternate HTML to suspicious sessions;
5. visible state transitions (loading, paused, playing, error overlays) require visual confirmation.

This is why browser-assisted extraction is not optional in this domain.

2.7 Page taxonomy with examples, inputs, and outputs

The system works on four page types.

2.7.1 `landing_page`

Role: discover candidate hosting pages and match-level context.

Typical signals: match cards or rows, team name pairs, date/time tabs, score indicators, links containing “matches”, “live”, or “events” path segments.

Expected outputs: ranked list of hosting-page URLs, extracted match metadata (teams, leagues, broadcasters), navigation evidence (screenshots, link structure).

2.7.2 `host_page`

Role: activate servers/players and extract stream evidence from hosting context.

Typical signals: single-match focus, server-switch buttons (serv1/serv2/serv3 labels), a visible player container, embedded iframe with albaplayer/clappr/hls.js, beIN Sports logo watermark.

Expected outputs: stream manifests (m3u8/mpd), per-server evidence, iframe embed URL for fallback, screenshots confirming active play state.

2.7.3 `embed_video_page`

Role: extract streams from iframe-centered player pages and fallback contexts.

Typical signals: minimal page with single player occupying most of the viewport, no navigation or ads, play button in center, network requests include `.ts` segments.

Expected outputs: final stream manifests after play activation, player-state diagnostics, iframe content diagnostics.

2.7.4 `other`

Role: filter noise pages (ads, unrelated redirects, irrelevant content) to protect runtime budget.

Page type	Typical input state	Expected output
`landing_page`	URL with listing layout, cards/rows, date tabs, multilingual labels	ranked hosting candidates, optional match metadata, navigation evidence
`host_page`	URL with one match context, server switchers, player container	streams (or strong candidates), server-level evidence, screenshots
`embed_video_page`	URL/frame containing reduced player UI and runtime media logic	final stream manifests/files, player-state diagnostics, iframe diagnostics
`other`	URL with non-investigative role	filtered decision with reason for skip/deprioritization

Table 2.2: Page-type input/output expectations used by the pipeline.

2.8 From the n8n phase to the new Python project

The company initially requested an n8n-based phase to validate feasibility quickly on real targets. That phase successfully confirmed the concept, but it also exposed limitations that became critical at scale:

- memory was mostly local to one workflow execution;
- long end-to-end flows were hard to keep stable when many hosting pages/servers were involved;
- parallel branches increased operational complexity and recovery difficulty;
- large workflows became difficult to maintain and reason about as prompts/tools evolved.

This is why the work moved to the new Python-based implementation with explicit memory, stronger observability, better runtime control, and clearer engineering boundaries.

2.9 Difficulties faced and what was done

2.9.1 Nested iframes and frame ambiguity

Difficulty. Useful players were often hidden several levels deep, and top-level signals were misleading.

Handling. Frame-tree inspection plus iframe diagnostics, combined with targeted inter-

actions and post-action re-inspection.

2.9.2 Popups, overlays, and click traps

Difficulty. Blocking overlays interrupted server switching and fake controls diverted the run.

Handling. Fallback click strategy with text matching, visibility checks, scroll-and-click, and JavaScript click as final fallback.

2.9.3 Tokenized and hidden stream paths

Difficulty. Some pages did not expose a direct `m3u8` in initial HTML; stream access was generated after runtime token exchange.

Handling. Stream extraction was moved to runtime evidence layers: CDP request/response monitoring, player API inspection, performance resource scans, and post-activation harvesting.

2.9.4 Anti-bot pressure

Difficulty. Repetitive automation patterns triggered challenges and degraded extraction continuity.

Handling. Normal-user behavioral pacing, reduced automation signals, adaptive retries, and evidence-first fallback when full extraction was blocked.

2.10 How target websites intentionally break extraction

Across both site families, the following break tactics were observed repeatedly:

- multi-step redirect chains designed to open popups before revealing the real player;
- nested iframe delegation across different domains to hide where the final media request originates;
- short-lived signed stream URLs (token + expiry) generated only after user-like interaction;
- bait resources and noisy assets that look important but are unrelated to the playable stream;

- anti-bot challenge pages that change response content when automation signals are detected.

Break tactic	Typical symptom in run traces	Agent/tool response
Popup-first flow	click opens ad tab and original page remains blocked	close extraneous targets, return focus, continue interaction on original page
Iframe nesting	top frame appears idle while playable state exists deeper	frame-tree diagnostics plus targeted frame-level actions
Tokenized manifests	no direct `m3u8` in HTML or first requests	activate player first, then harvest runtime network/player evidence
Challenge gating	alternate content, delayed controls, or interstitial loop	human-like pacing, retry strategy, and evidence-preserving fallback

Table 2.3: Observed break tactics and corresponding navigation strategy.

2.11 Screenshot storage: Cloudinary CDN

Each agent calls `screenshot()` after key state transitions. The resulting images serve as primary evidence: they confirm player activation, show server labels, and provide human-auditable context for any extracted stream URL.

Screenshots are stored using the **Cloudinary** media CDN. After Puppeteer captures a PNG buffer, the tool uploads it to Cloudinary via their Upload API and receives back a persistent URL. This URL is stored alongside the run event in PostgreSQL and displayed in the operator console.

Why Cloudinary rather than local storage?

- screenshots are immediately accessible from the operator console without needing to serve files from the backend;
- Cloudinary applies automatic compression and resizing via URL parameters, keeping evidence lightweight without losing diagnostic quality;
- the CDN URL is stable and can be embedded in takedown emails as external evidence links;
- it decouples screenshot storage from the backend container, simplifying cloud deployment where ephemeral container storage would otherwise lose evidence.

2.12 Ad blocking with uBlock Origin

Illegal streaming sites monetize through aggressive advertising. This creates two problems for automated extraction:

1. popups and interstitial ads occupy new browser contexts and divert the agent's interaction away from the real player controls;
2. redirect chains triggered by ad click events can leave the original page in an unusable state.

The browser profile uses `uBlock Origin` as the main preventive layer, while the agents keep explicit popup-recovery behavior for cases where extension-level filtering is not sufficient.

2.12.1 uBlock Origin

uBlock Origin operates as a Chromium extension loaded at browser launch. It intercepts network requests at the extension level, blocking ad server domains, tracking pixels, and ad-injecting scripts before they execute. This prevents the majority of popup-opening ad scripts from running at all.

Configuration points tuned during the internship:

- filter list selection: EasyList + EasyPrivacy as the baseline, supplemented by regional filter lists relevant to Arabic streaming domains;
- dynamic filtering: specific CDN domains used for player content were *whitelisted* to prevent false positives that blocked player assets;
- the extension was loaded via `-load-extension` and `-disable-extensions-except` Puppeteer flags to avoid fingerprint signals.

Limitation of extension filtering. No extension is a complete solution. Sites that serve ad content from the same domain as player content cannot be filtered without breaking the player. This is why the hosting and embedded agents retain explicit popup handling logic (detect new tab → close → retry) as a fallback even when ad-blocking is active.

Layer	Primary mechanism	Coverage
uBlock Origin	Network request interception at extension level	Ad servers, tracking pixels, injected ad scripts
Popup handler (agent)	Detect new browser context and close it	Same-domain ads that extensions cannot filter

Table 2.4: Ad containment strategy used in the platform after removing the secondary extension layer.

2.13 Techniques used inside tools (Puppeteer and `evaluate()`)

Tooling techniques used repeatedly included:

- `page.evaluate(...)` for runtime DOM/player state extraction;
- network interception and response metadata inspection;
- frame-aware interactions and diagnostics;
- screenshot capture after each critical transition;
- retroactive resource checks through performance APIs.

2.14 Continuous tuning as an operating requirement

The report treats anti-bot adaptation and adblock tuning as continuous operations. The environment changes too often for a one-time technical setup.

Challenge	Typical symptom	Applied handling	Remaining risk
Nested iframe chains	wrong frame selected for interaction	frame diagnostics + iterative targeting	frame structure can change per session
Popup blockers	click target unreachable	fallback click strategy	some overlays need custom site logic
Tokenized stream generation	no final stream in static source	post-activation network/player harvesting	token lifetime can expire quickly
Cloudflare-like challenge	interstitial/challenge screen	low-signal behavior + controlled retries	intermittent manual review still required

Table 2.5: Difficulty-to-handling mapping used in practice.

2.15 Streaming protocol infrastructure

Understanding the protocols used by streaming piracy infrastructure is a prerequisite for understanding what the platform is designed to capture. Piracy hosts do not use a single streaming technology: content is served over several adaptive-bitrate protocols, each with its own manifest format, URL structure, and tokenization scheme. The `harvest` tool is built to intercept all of them simultaneously.

2.15.1 HLS — HTTP Live Streaming

HLS [22] is the dominant adaptive-bitrate protocol encountered in this domain. It is text-based and organized in a strict two-level hierarchy.

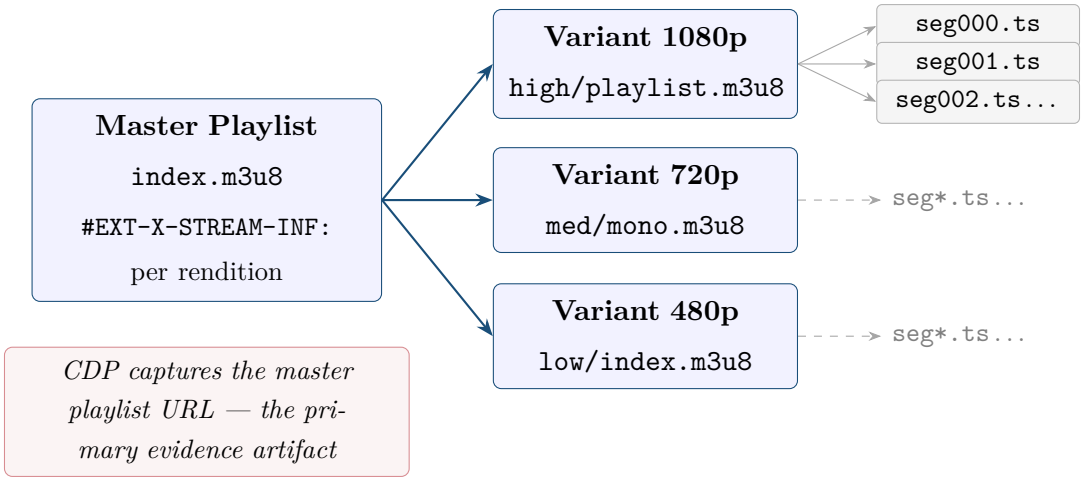

Figure 2.2: HLS two-level playlist hierarchy. The `harvest` tool captures the master playlist URL at the CDP network layer; variant playlists and `.ts` media segments are derived from it by the player.

The **master playlist** (commonly named `index.m3u8`, `manifest.m3u8`, or `playlist.m3u8` depending on the CDN) lists every available rendition with its bitrate, resolution, and codec. Each rendition entry points to a **variant playlist** that contains time-coded references to the actual MPEG-2 Transport Stream media segments (`.ts` files). The player selects the appropriate variant based on available bandwidth.

From an evidence perspective, the master playlist URL is the most important artifact because it is the entry point that ties the stream to a specific CDN path. Several URL patterns appear regularly across piracy infrastructure:

- `/live/index.m3u8` — the canonical HLS master playlist;
- `/stream/mono.m3u8` — a single-rendition playlist that skips the master level; the player goes directly to segments;

- `/hls/playlist.m3u8?token=`*`xxx`*`&expires=`*`timestamp`* — a tokenized manifest where signed query parameters grant short-lived CDN access;
- `/live/manifest.m3u8` — an alternative naming convention common on infrastructure built around open-source re-streaming panels.

Tokenized manifests are the most operationally significant variant. The token in the query string is signed by the CDN key server and carries an expiry timestamp. The `harvest` tool captures the full URL including the token at the moment the browser requests it, preserving complete evidence before the token expires.

2.15.2 MPEG-DASH

MPEG-DASH (Dynamic Adaptive Streaming over HTTP, ISO/IEC 23009-1) is the second protocol regularly encountered. Unlike HLS, DASH manifests are XML documents (`.mpd`) that describe a hierarchical segment addressing scheme:

- **MPD** (Media Presentation Description) — the root XML manifest;
- **Period** — a contiguous time range of the presentation;
- **AdaptationSet** — a group of representations sharing the same media type (video, audio, or subtitles);
- **Representation** — one specific quality level with its codec and bitrate;
- **SegmentTemplate** or **SegmentList** — the addressing scheme for individual `.m4s` fragment files.

The `harvest` tool captures DASH manifests via the URL pattern `/\.mpd(\?|$)/i`, which matches naming conventions such as `manifest.mpd`, `stream.mpd`, and `dash.mpd`.

2.15.3 Additional protocols captured

Beyond HLS and DASH, the `harvest` tool captures three further protocol families:

- **Progressive MP4** (`.mp4`) — static download links embedded in player configuration objects; less common on live streams but present on replay and clip pages.
- **WebM** (`.webm`) — the Chromium-native open container format, used on some alternative-player implementations and lower-quality fallback streams.
- **Smooth Streaming** (`.ism/manifest`) — the Microsoft-origin fragmented-MP4 format, occasionally found on infrastructure using legacy streaming servers.

2.15.4 Protocol comparison

Attribute	HLS (RFC 8216)	MPEG-DASH (ISO 23009-1)	Progressive / Other
Manifest format	Plain text (`.m3u8`)	XML (`.mpd`)	None (direct URL)
Segment format	MPEG-TS (`.ts`) or fMP4 (`.m4s`)	fMP4 (`.m4s`)	MP4, WebM
Common patterns	`index.m3u8`, `mono.m3u8`, `playlist.m3u8`	`manifest.mpd`, `stream.mpd`	`video.mp4`, `clip.webm`
Tokenization	Query string (`?token=&expires=`)	Query string or signed path	Signed path or none
Hierarchy levels	2 (master → variant → segments)	4 (MPD → Period → AdaptationSet → segments)	1 (direct)
CDP capture layer	`Network.requestWillBeSent` + perf API	`Network.requestWillBeSent`	`Network.requestWillBeSent`

Table 2.6: Comparison of the streaming protocols captured by the `harvest` tool, with their manifest formats, URL patterns, and detection method.

2.15.5 The harvest tool's six-layer capture strategy

The `harvest` tool uses six parallel detection layers to ensure manifest URLs are captured regardless of which protocol is in use and regardless of whether the manifest loaded before the tool was invoked:

1. **CDP `Network.requestWillBeSent`** — the Chrome DevTools Protocol fires this event for every outgoing network request, including from within cross-origin iframes. This is the primary layer and fires before the request leaves the browser.
2. **Puppeteer response listener** — a secondary `page.on('response')` handler provides redundancy and captures response HTTP status codes alongside the URL.
3. **DOM element scan** — the tool queries `<video src>` and `<source src>` attributes for statically embedded stream references.
4. **JavaScript player API inspection** — `page.evaluate()` probes known player surfaces: hls.js instance URLs, video.js current source, jwplayer playlist items, and dash.js manifest references.
5. **`performance.getEntriesByType('resource')`** — a retroactive resource-timing

scan recovers manifest URLs loaded before the tool began listening.

6. **Frame-level CDP session** — when the player is in a cross-origin iframe, a dedicated CDP session is opened on that frame to capture its network traffic independently.

All captured URLs are matched against the protocol patterns and returned grouped by protocol. The concrete implementation is detailed in Chapter 5.

2.16 Remaining limitations of rule-based approaches

Even with the mitigations above, difficult websites remain. This justifies the fallback role of the agentic path:

- some protections still force partial evidence outcomes;
- highly dynamic pages can invalidate previously successful selectors quickly;
- operational cost grows with deeper exploration and server count.

For this reason, the adaptive workflow is used where its flexibility provides clear recovery value.

With the technical background established — the domain challenges, the tools selected to address them, the streaming protocols being targeted, and the remaining limitations that motivate adaptivity — Chapter 3 formalizes the methodology, requirements, and design constraints that guided the system's architecture.

CHAPTER 3

Methodology, Requirements Analysis, and Design Constraints

This chapter presents the methodological frame adopted for the internship, explains why Design Science Research Methodology (DSRM) was selected, maps the six DSRM phases to the concrete work carried out, and derives the requirements and constraints that shaped the artifact described in Chapters 4 and 5. Because the internship delivered and evaluated technical artifacts rather than a predictive model, the methodological emphasis is on artifact design, demonstration, and evaluation.

3.1 Methodology options considered

The internship combined software engineering, browser automation, prompt design, and evidence-oriented investigation. Three methodological options were considered: Agile, CRISP-DM, and Design Science Research Methodology (DSRM).

3.1.1 Agile

Agile practices matched the day-to-day execution rhythm of the internship. Short cycles of implementation, run inspection, failure analysis, and prompt or tool revision were essential because illegal streaming sites change quickly and often expose new obstacles only during live testing.

3.1.2 CRISP-DM

CRISP-DM [13] offers a clear sequence from business understanding to deployment and evaluation. Parts of that logic were useful for organizing observations and reporting progress. However, the project was not a classical data-mining lifecycle and did not aim to optimize a predictive model on a labeled dataset.

3.1.3 Design Science Research Methodology (DSRM)

Design science research focuses on the creation and evaluation of artifacts that solve relevant organizational problems [14, 15]. This framing matches the internship more closely because the central deliverable is an engineering artifact: an agentic browser-based evidence-collection pipeline composed of prompts, tools, browser traces, screenshots, structured outputs, and evaluation metrics.

Methodology	Strength for this project	Limitation for this project
Agile	Excellent short-cycle execution and adaptation during prompt, tool, and browser refinement	Less explicit about how to justify and evaluate a technical artifact as a research contribution
CRISP-DM	Useful discipline for scoping, evaluation, and deployment discussion	Dataset/model-centered framing is not the main logic of this internship
DSRM	Artifact-centered framing that explicitly links problem, design, demonstration, and evaluation	Needs complementary iteration rhythm for fast engineering feedback

Table 3.1: Methodology comparison for the internship project.

3.2 Chosen methodology: DSRM with iterative prototyping

DSRM was adopted as the overall methodology, while Agile-style short iterations were used inside each phase to refine prompts, tools, and browser behavior based on observed failures.

DSRM fits the project better than CRISP-DM for three reasons. First, the project is artifact-centered, not dataset-centered or model-centered: the main work was to design a reliable evidence-collection system under adversarial web conditions. Second, the main

output is a working and proposed system architecture together with a validated agent workflow, not a trained predictive model. Third, evaluation focuses on tool behavior, evidence quality, trace completeness, extraction success and failure patterns, cost, and robustness rather than on statistical accuracy against a fixed labeled benchmark.

CRISP-DM remains a useful secondary lens for discussing operational context and evaluation discipline, but it is not the primary methodological backbone of the report. Figure 3.1 summarizes the adopted DSRM view.

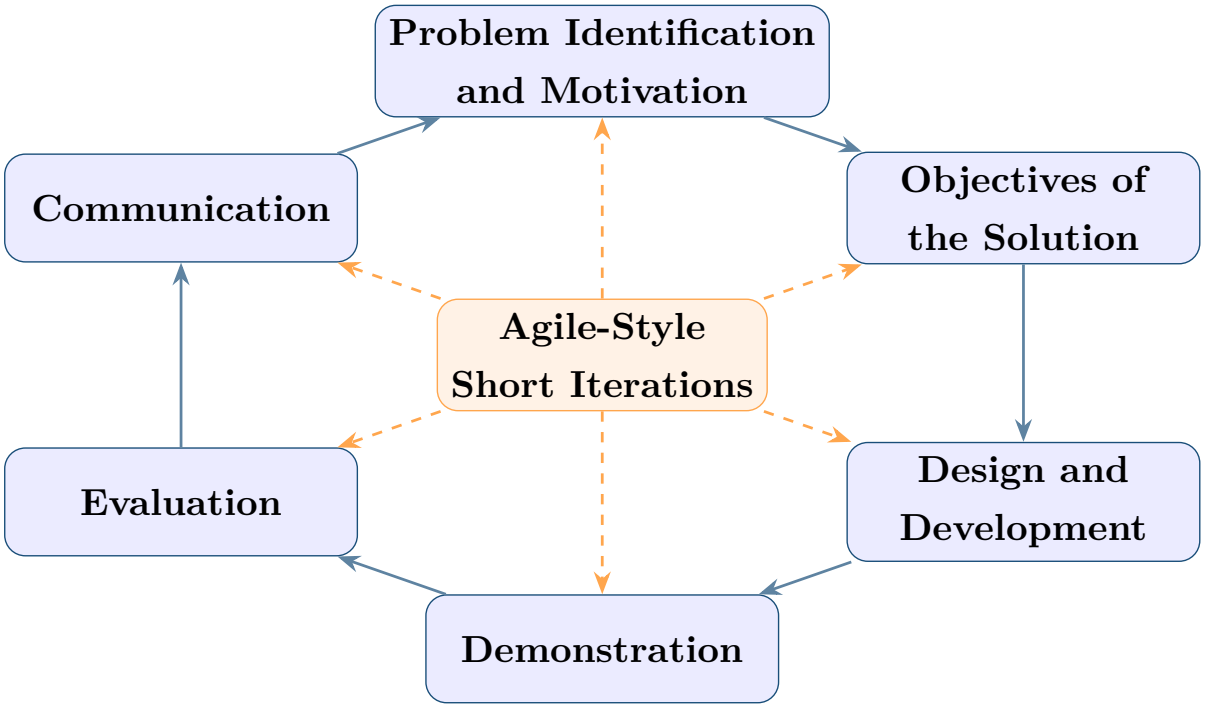

Figure 3.1: DSRM adopted for the project, with Agile-style short iterations inside each phase.

3.3 DSRM applied to this project

Following Peffers et al. [15], the project was organized around six DSRM phases. Table 3.2 maps each phase to the concrete internship work and its main outputs.

DSRM phase	Application in this project	Main output
Problem identification and motivation	Illegal streaming evidence collection is dynamic, adversarial, multilingual, and difficult for static scrapers because evidence may appear only after runtime interaction, iframe traversal, or player activation.	Problem framing and motivation for the artifact
Objectives of the solution	Define that the artifact must classify pages, extract host/embed/stream evidence, capture screenshots, preserve traces, and calculate evaluation metrics that support reviewable evidence collection.	Solution objectives and acceptance criteria
Design and development	Build the n8n prototype and the validated single-agent workflow used in the company context; derive a Python/FastAPI reference architecture for modularity, observability, memory, evaluation, and future scalability.	Operational workflow plus reference architecture
Demonstration	Run the artifact on representative streaming-site families: landing pages, hosting pages, embedded player pages, and iframe/popup/tokenized-stream cases.	Demonstration cases and observed behaviors
Evaluation	Measure prompt behavior, tool usage, extraction success and failure, token and cost metrics, cache behavior, trace quality, and practical limitations.	Evaluation traces, metrics, and findings
Communication	Produce the internship report, diagrams, architecture documentation, and a future deployment roadmap for industrialization.	Communicated artifact and future roadmap

Table 3.2: Application of DSRM to the evidence-collection project.

3.4 Project evolution timeline

The internship evolved from early n8n validation to a company-side single-agent deployment, then to the Python reference architecture, evaluation campaigns, and a future target cloud deployment. Figure 3.2 summarizes this sequence.

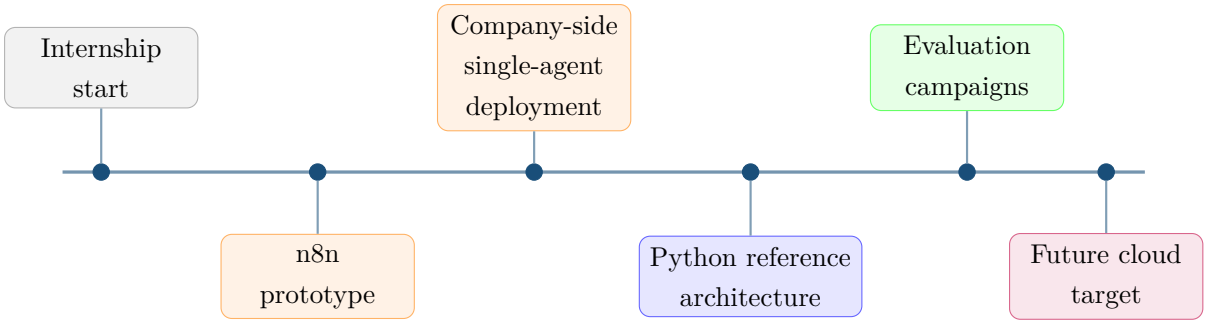

Figure 3.2: Project evolution timeline from n8n validation to Python reference architecture and future deployment target.

The validated n8n workflow established the operational baseline used in the company context. The Python reference architecture should be interpreted as the maintainability-oriented evolution of the same evidence-collection logic, while the future cloud target remains a planned deployment direction rather than a claim of full platform replacement during the internship.

3.5 Functional and non-functional requirements

The requirements were addressed across two complementary levels. At the operational level, the validated n8n workflow covered the core evidence-collection tasks used in the company context. At the reference-architecture level, the Python implementation generalized those tasks into a more maintainable design with stronger observability, memory, evaluation support, and future scalability.

Table 3.3 summarizes the ten functional requirements derived from the operational context.

ID	Requirement summary	Priority
FR-1	URL ingestion and run lifecycle management	Critical
FR-2	Page-type classification with DOM and screenshot signals	Critical
FR-3	Specialist routing by page type	Critical
FR-4	Tab-aware landing-page candidate discovery	High
FR-5	Multi-server hosting extraction loop	Critical
FR-6	Embedded player activation and stream capture	Critical
FR-7	Hosting-to-embedded fallback	High
FR-8	Screenshot capture and persistent evidence storage	High
FR-9	WHOIS / IP infrastructure enrichment	Medium
FR-10	Operator review surface and trace inspection	High

Table 3.3: Functional requirements summary with priority.

Table 3.4 lists the non-functional requirements and the design choices used to address them across the validated workflow and the Python reference architecture.

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	n8n validated behavior; Python modular agents, versioned prompts, and tool registry
NFR-2	Observability	Structured event log, evaluation database, SSE review surface, and trace persistence
NFR-3	Cost control	Model routing, provider-side caching, and token budgets
NFR-4	Reliability	Tool error contracts, popup handling, and bounded retry logic
NFR-5	Scalability	Target Azure Container Apps Jobs execution with Azure Service Bus queueing
NFR-6	Portability	Docker Compose, configurable browser profiles, and separable worker services
NFR-7	Evidence traceability	Run-linked artifact chain: event → screenshot → structured output

Table 3.4: Non-functional requirements and implementation approach.

3.6 Design constraints

From a design science perspective, the artifact had to satisfy the problem environment while remaining evaluable and maintainable. Six constraints bounded the design space:

1. target-site layouts and anti-bot behavior can change without notice between broadcast windows;
2. the player can be nested behind multiple iframe hops from the top-level page;
3. popups, overlays, and fake play controls can divert scripted extraction;

4. final stream manifests may only be generated after a runtime token exchange triggered by player activation;
5. inference cost scales with exploration depth, the number of hosting pages, and the number of servers tested per page, requiring active caching and model routing;
6. the Python reference architecture must remain decoupled from enterprise production databases, so persistence in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs.

3.7 Prompting methodology evolution: intent-based to ReAct-style loops

The prompting strategy evolved during the DSRM design-and-development and evaluation phases. The first version relied on high-level intent prompts, which often produced premature answers on ambiguous pages. The later version adopted a ReAct-style loop in which the agent had to observe, reason, act, and re-check the browser state before declaring success. Figure 3.3 compares the two approaches side by side.

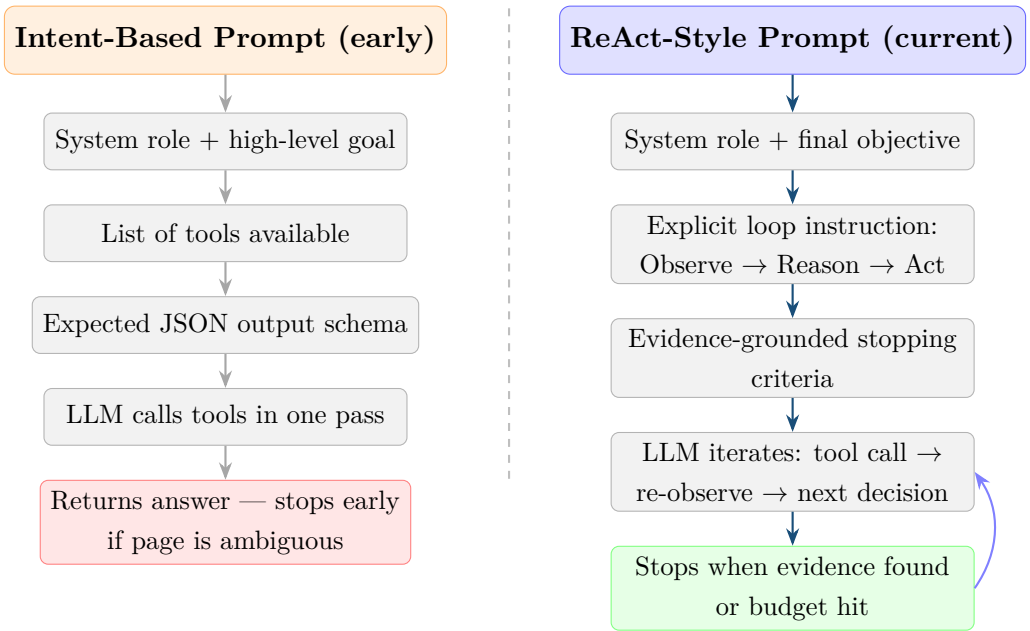

Figure 3.3: Prompt strategy evolution: intent-based linear flow versus ReAct-style iterative loop.

Why ReAct performed better. ReAct-style behavior:

- reduced early stopping on ambiguous pages where the first observation was insufficient;

- enforced evidence-driven step progression, so the agent could not declare success without a browser-grounded confirmation step;
- made tool traces easier to audit because each reasoning step was explicitly logged before and after the action.

3.8 Derived design principles

The methodology, constraints, and evaluation results led to the following design principles:

- classify page role before deep extraction;
- separate operational workflow validation from reference-architecture generalization;
- ground every conclusion in browser evidence rather than textual guesswork;
- preserve auditable traces, screenshots, and structured outputs for every critical step;
- use specialist agents and explicit fallback paths rather than one unconstrained agent;
- optimize robustness together with cost through caching, budgets, and model routing;
- design the future cloud target around loose coupling between API services, browser workers, observability storage, and artifact storage.

Taken together, the DSRM framing, the two-level requirement view, and the derived design principles establish the scope of the artifact presented in this report. Chapter 4 translates these inputs into a concrete reference architecture, documenting the component structure, agent roles, tool allocation decisions, and target deployment model.

CHAPTER 4

System Architecture

4.1 Overview of the Python Reference Architecture

This chapter presents the Python reference architecture designed after the validated n8n workflow. The validated n8n workflow served as the operational validation path, while the architecture described here formalizes the same evidence-collection logic into clearer layers and runtime boundaries for maintainability, observability, evaluation, and future deployment.

Figure 4.1 shows the logical sequence followed by one investigation run, from the input URL to the validated evidence bundle. This view is independent of concrete deployment technology and focuses on decisions, evidence sources, and outputs.

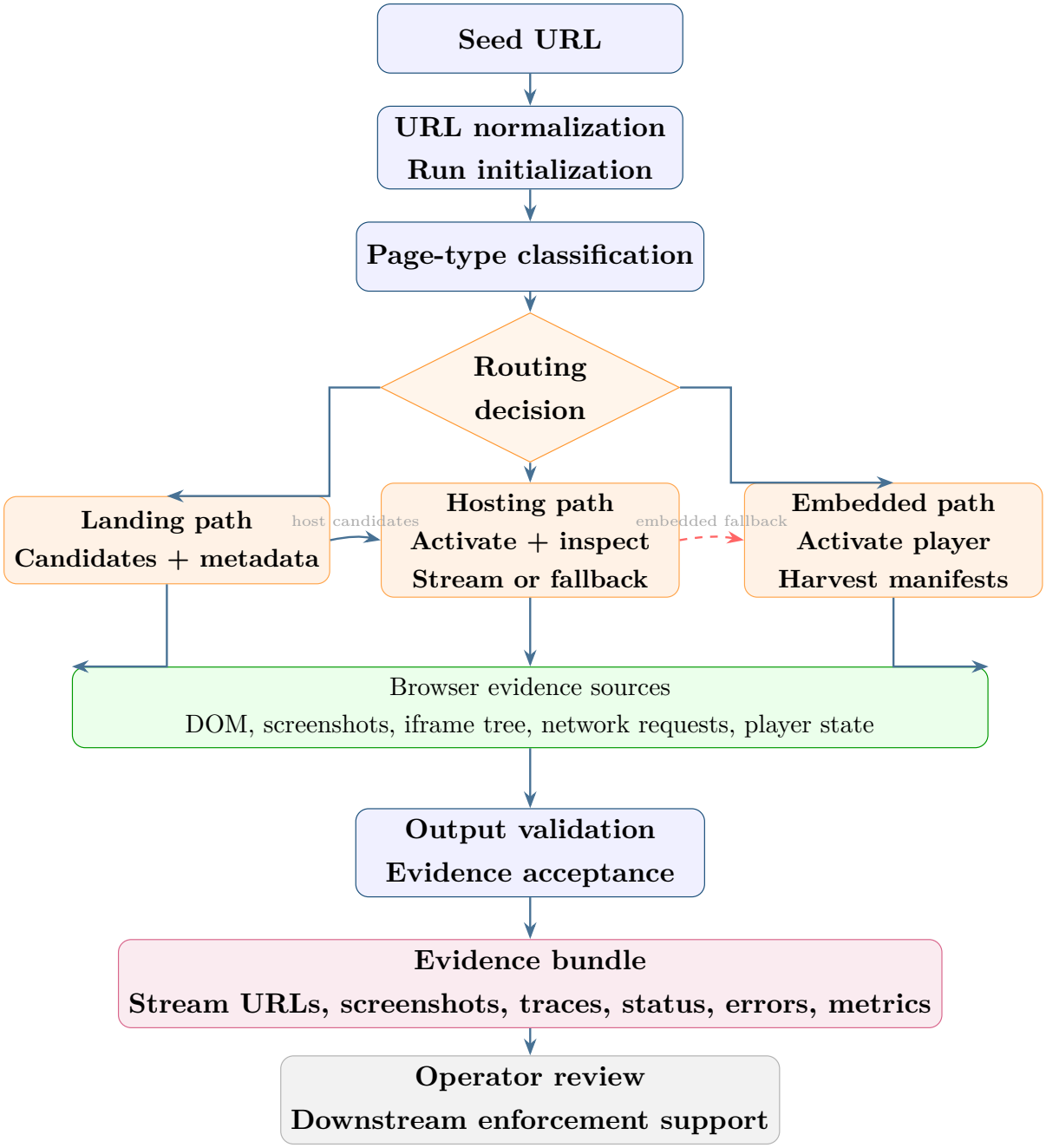

Figure 4.1: Logical workflow of an evidence-collection run.

In this logical view, the landing-page path discovers host-page candidates, the host-page path attempts direct player activation and stream extraction, and the embedded path acts either as the direct route for embedded players or as the fallback when hosting-page extraction yields only iframe-based evidence. The evidence-source block is kept separate so that DOM observations, screenshots, iframe structure, network activity, and player state appear as validation inputs rather than as implementation technologies.

4.2 Architectural layers

4.2.1 Execution and API layer

The FastAPI backend is the entry point for runs, status checks, traces, and evaluation routes. It isolates runtime control from browser internals and gives the operator console a stable interface. Key endpoints include:

- `POST /runs` — submit a seed URL and start an investigation run;
- `GET /runs/{id}/events` — stream live run events;
- `GET /runs/{id}/artifacts` — retrieve stream evidence and screenshots;
- `GET /evaluate` — access the evaluation and review surface.

4.2.2 Agent orchestration layer

The orchestrator coordinates execution order and specialist handoff. It is intentionally lightweight so routing logic stays understandable and model-intensive work stays in specialist agents. The orchestrator calls the classification agent first on every URL, then dispatches to the appropriate specialist based on the classification result. Agents do not call one another directly: the orchestrator forwards structured handoff data built from tool outputs, screenshots, and the remaining run budget.

4.2.3 Specialist-agent layer

The specialist layer contains four focused roles. Each agent has its own system prompt, tool set, and stopping criteria. This structure reduces over-general prompts and makes failure analysis more precise. Agent composition is detailed in Chapter 5.

4.2.4 Browser tool layer

The browser tool layer exposes controlled actions (inspect, navigate, click, screenshot, harvest). It acts as the boundary between model decisions and browser-side execution. All tools are implemented as Node.js Puppeteer processes invoked from the Python backend.

4.2.5 Anti-evasion and runtime-hygiene layer

A dedicated anti-evasion perspective is required in this domain. The architecture embeds runtime hygiene policies that were validated during the internship:

- realistic browsing behavior and pacing;
- fingerprint profile hardening and periodic adjustments;
- adaptive adblock behavior rather than static one-time setup;
- interaction fallbacks for overlays and unstable selectors;
- evidence-first operation (screenshots + network + DOM state).

4.2.6 Persistence, memory, and observability layer

This layer stores runs, events, tool calls, LLM calls, screenshots, errors, metrics, and structured outputs. It enables post-mortem analysis and supports both short-term and long-term memory strategies. The Python architecture does not depend on the enterprise production database. The database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs.

4.2.7 Operator interface layer

The Next.js operator console provides run launch, live tracing, historical inspection, and evaluation navigation. It is designed for operational review, not only developer debugging.

To keep this chapter focused on the views that are most useful for implementation and review, the report emphasizes the logical workflow, component, physical runtime, memory, and target cloud deployment views. Together, these diagrams explain how one investigation run progresses, how the software modules connect, and how the Python reference architecture can be deployed honestly.

4.3 Component architecture

Figure 4.2 shows the concrete software modules that implement the Python reference platform and the main interfaces between them.

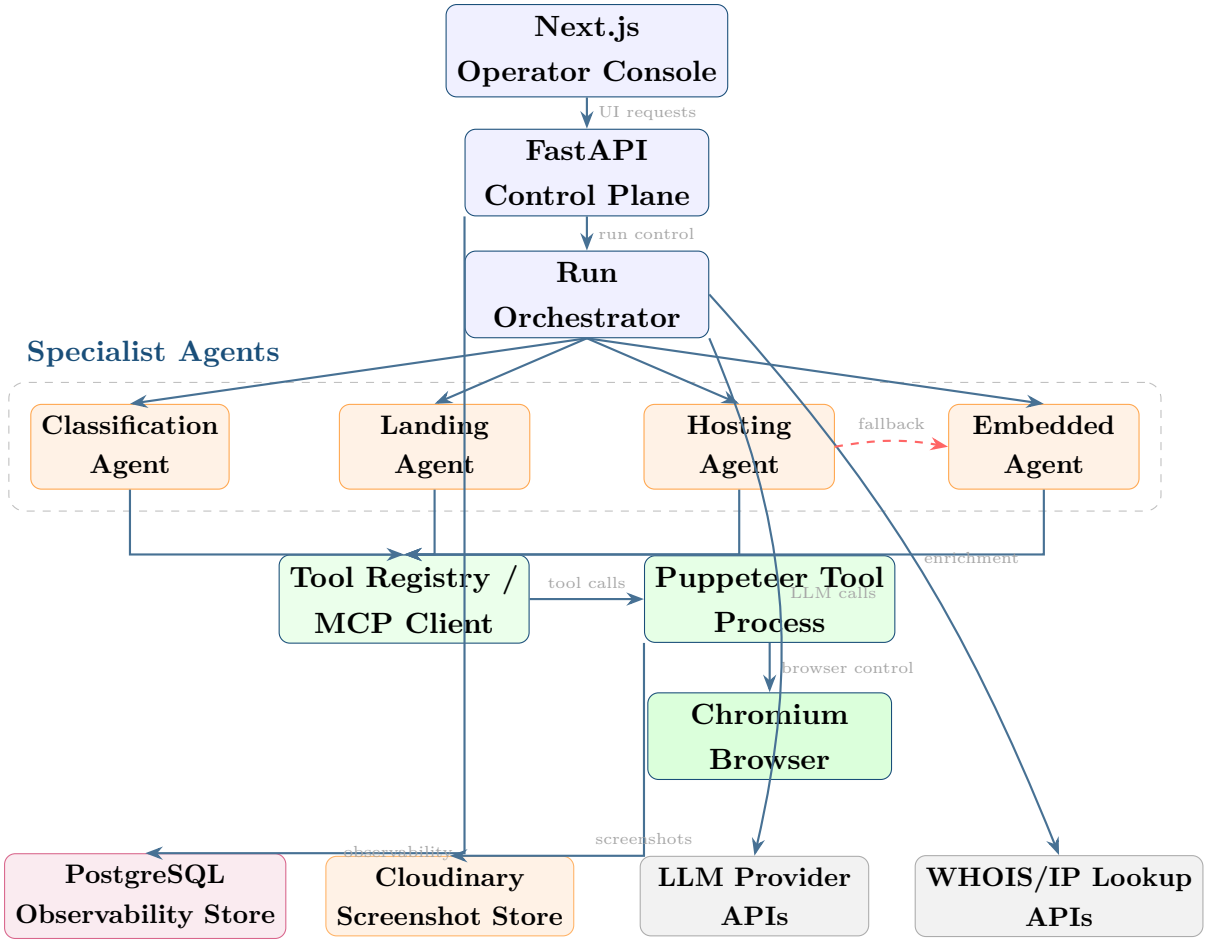

Figure 4.2: Component architecture of the Python reference platform.

The component view makes the concrete software boundaries explicit: the operator console and control plane initiate runs, the orchestrator coordinates specialist agents, the Tool Registry / MCP Client standardizes browser-tool access, the Puppeteer process drives Chromium, PostgreSQL stores observability data, Cloudinary stores screenshots, and the LLM plus WHOIS/IP services remain external dependencies.

4.4 Physical and Deployment Architecture

Figure 4.3 shows the physical runtime placement of the main services used to execute and review the Python reference architecture. Unlike the logical workflow and component views, this figure emphasizes where the UI/API service, Python agents, browser automation, persistence, and external services sit at runtime.

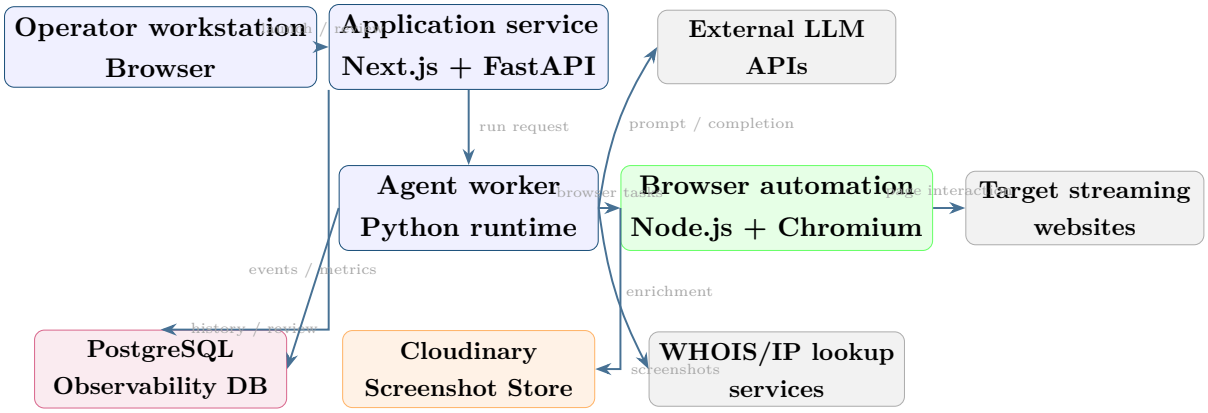

Figure 4.3: Physical runtime view of the Python reference architecture.

This runtime view separates operator access, application control, Python orchestration, and browser automation. It also keeps screenshots and observability data outside the browser process so that evidence survives container restarts and remains reviewable after execution.

4.4.1 Target Cloud Deployment Architecture

The target cloud deployment keeps the same logical roles but introduces queue-driven job boundaries between request intake and browser-heavy execution. This is a target cloud deployment view rather than the main runtime used during the internship.

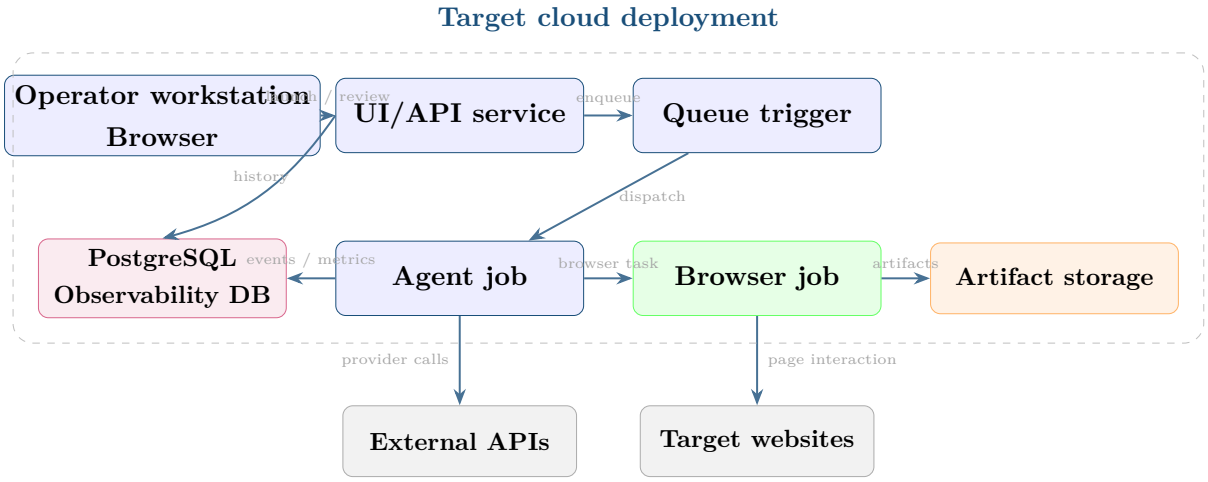

Figure 4.4: Target deployment architecture for future queue-driven extraction jobs.

During the internship, the company-side operational deployment remained the validated n8n single-agent workflow. The Python architecture was evaluated as a reference implementation and designed to remain compatible with future Azure Container Apps Jobs execution.

4.5 Memory and knowledge reuse architecture

4.5.1 Run-level (short-term) memory

Run-level memory captures local progress for the active session: visited pages, tried actions, discovered signals. It reduces repeated failures in the same run by making previous tool call results available to the agent in the same conversation context.

4.5.2 Cross-run (long-term) memory

Cross-run memory stores reusable site knowledge between runs. It supports faster warm starts on recurring domains by reusing successful selectors, layout patterns, and server strategies. The memory is implemented as a structured database keyed by domain and page-type.

Memory type	Scope	What is stored
Short-term (run-level)	One investigation run	Visited URLs, tool call results, agent reasoning trace, discovered signals within current session
Long-term (cross-run)	Across multiple runs	Successful selectors, layout fingerprints, server count patterns, known provider domains, observed break tactics per domain

Table 4.1: Two-layer memory architecture and stored content.

4.6 Caching and cost-aware architecture

Cost control is part of the architecture, not an afterthought. The main mechanisms are:

- provider-aware prompt caching (LLM system prompts cached across turns);
- tool-result reuse when observations are identical within a run;
- specialist routing to avoid unnecessary heavy steps on simple cases;
- per-run cost visibility in observability surfaces.

4.7 Architectural transition from the prototype

The transition from the validated n8n workflow to the Python reference architecture changed the architecture in three practical ways:

- prototype-friendly visual orchestration moved to explicit code boundaries;
- short-lived workflow context moved to persistent memory and observability;
- ad hoc experimentation moved to repeatable engineering surfaces with tests and review tooling.

Aspect	n8n workflow	Python reference architecture
Orchestration	Visual workflow editor	Explicit Python orchestrator
Memory	Per-workflow context only	Short-term + cross-run observability database
Observability	Limited aggregate logs	Per-event structured traces with cost
Prompt lifecycle	Inline node text	Versioned system prompts per agent
Testing	Manual trigger only	Automated regression tests + eval routes
Cost visibility	Basic	Per-run token/cost tracking

Table 4.2: Architectural changes from the validated n8n workflow to the Python reference architecture.

This transition documents how the validated workflow can evolve into a more maintainable and observable evidence-collection system.

CHAPTER 5

Implementation

5.1 Implementation trajectory

Implementation progressed in two explicit stages:

- **Stage 1: n8n phase** requested by the company for rapid feasibility validation;
- **Stage 2: Python reference architecture** for maintainability, stronger memory, clearer observability, systematic evaluation, and future long-run execution.

The second stage keeps the lessons of the first stage while generalizing them into an academic and engineering reference implementation rather than claiming a full replacement of the operational company deployment.

5.2 Backend and operator surfaces

Within the Python reference architecture, the FastAPI backend acts as a control plane for runtime coordination, persistence, and operator-facing services. It is not presented here as a deployed enterprise production API, but as the academic implementation used to organize the reference workflow around explicit interfaces. In practice, it handles runtime health and preflight checks, run launch and workflow execution, background jobs, run status inspection, trace and event persistence, screenshot and evidence retrieval, provider and runtime configuration, pricing configuration, evaluation execution, tool playground support, and database-backed operator views.

The Next.js frontend complements this backend as an operator and reviewer interface rather than as a passive visual dashboard. The console supports run launch, readiness checks, live trace review, historical run inspection, evaluation workflows, settings management, and controlled access to stored observability records. Live run progress is streamed

through Server-Sent Events (SSE), allowing operators to watch tool calls, agent transitions, and evidence capture as they happen.

5.2.1 Backend runtime and API surface

The backend groups related capabilities into route families rather than exposing one monolithic execution endpoint. Table 5.1 summarizes the main interface families used in the Python reference architecture. The endpoint names are presented as representative examples of the implemented route families, not as a claim of a finalized enterprise API contract.

Route family	Example endpoints	Responsibility
Runtime health	`/health`, `/ui/browser/status`	Verify browser, MCP, and tool-profile readiness before launching runs.
Core pipeline execution	`/run`, `/extract`, `/ui/workflows/run`	Start a workflow or agent extraction run.
Run inspection	`/ui/runs/{run_id}`, `/ui/overview`	Display run status, traces, artifacts, streams, screenshots, and costs.
Evaluation	`/ui/evaluations/lab`, `/ui/evaluations/run`	Execute evaluation suites or manual website batches and summarize metrics.
Configuration	`/ui/config`, `/ui/providers/models`, `/ui/pricing`	Manage LLM provider, model settings, browser runtime, and pricing.
Tool playground	`/ui/tools/playground`	Test browser tools independently from full workflow runs.
Database inspection	`/ui/database/tables`	Inspect stored run, evaluation, and tool records through an allowlisted interface.

Table 5.1: Representative backend route families in the Python reference architecture.

This separation is useful for engineering reasons. Health and preflight routes reduce avoidable failed runs; execution routes isolate workflow launch from review concerns; inspection routes make traces and artifacts auditable after completion; and evaluation, configuration, and tool-playground routes support iterative improvement without forcing all debugging work through the main extraction path.

5.2.2 Operator console and frontend workflows

The Next.js frontend functions as an operational review surface for the reference architecture. Its purpose is to help an operator or evaluator initiate investigations, observe execution, inspect evidence, review failure modes, and adjust runtime settings. In this sense, the frontend is part of the workflow-control loop: it launches runs, checks runtime readiness, watches live events and traces, reviews completed runs, inspects extracted streams, screenshots, provider analysis, errors, and costs, launches evaluation batches, updates provider/model/runtime settings, and reviews evaluation results.

Frontend surface	Main function	Backend dependency
Overview dashboard	Summarizes runs, costs, success/failure, and active jobs.	`/ui/overview`
Run detail page	Shows events, tool calls, LLM calls, screenshots, and streams.	`/ui/runs/{run_id}`
Workflow launcher	Starts a new URL investigation.	`/ui/workflows/run`
Evaluation lab	Runs manual or configured evaluation suites.	`/ui/evaluations/run`
Settings/config page	Updates provider, model, pricing, and browser runtime settings.	`/ui/config`, `/ui/pricing`, `/ui/providers/models`
Database viewer	Reviews allowlisted persistence tables.	`/ui/database/tables`

Table 5.2: Main frontend workflows and their backend dependencies.

Taken together, the backend and frontend form a reviewable implementation surface for the Python reference architecture. The backend exposes structured control and persistence interfaces; the frontend turns those interfaces into repeatable operator workflows for launch, observation, diagnosis, evaluation, and configuration.

5.3 Four-agent design: specialist roles and boundaries

Each agent is designed around one clear responsibility. Figure 5.1 shows the four specialist agents with their tools and interfaces.

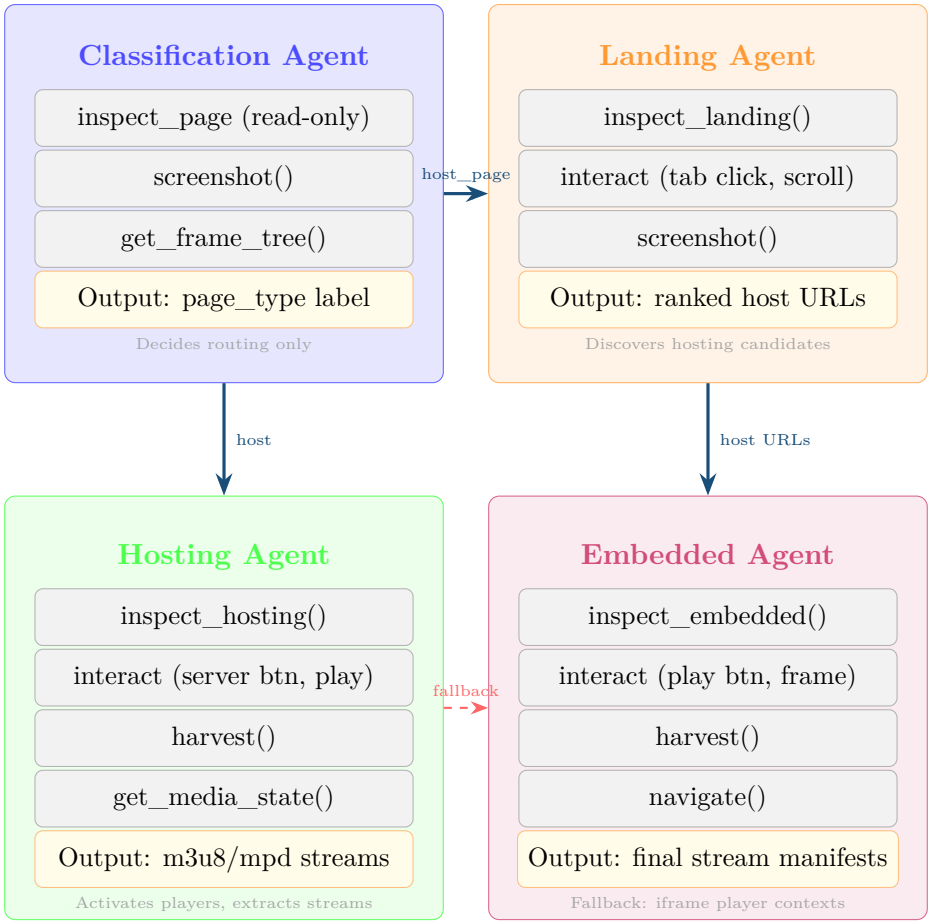

Figure 5.1: Four specialist agents: tools, outputs, and routing relationships.

5.4 Agent design: reasoning toward a final objective

Each agent follows the same operating principle: it has a final objective and a toolset to reach that objective through iterative evidence-based decisions.

- **Classification agent objective:** route the page correctly.
- **Landing agent objective:** discover and rank hosting candidates.
- **Hosting agent objective:** activate players, test servers, and extract streams.
- **Embedded agent objective:** recover streams from iframe-centered contexts and difficult fallback paths.

5.5 Prompt design and structure per agent

The full system prompts are omitted from this shortened report. In the main body, it is more useful to summarize what each prompt is responsible for and what constraint keeps it under control.

Agent	Prompt objective	Main control rule
Classification	Assign one page type from the predefined taxonomy using read-only evidence	No interaction; short tool budget; return a single structured decision
Landing	Discover and rank hosting candidates on listing pages	Iterate over tabs and dynamic listings, but stop when no new candidates appear
Hosting	Activate players, test servers, and recover stream evidence from host pages	Validate actions with screenshots or media state before harvesting; fall back when needed
Embedded	Recover final stream evidence inside iframe-centered player contexts	Use bounded play-attempt logic and return an explicit failure reason when evidence is absent

Table 5.3: Prompt summary per specialist agent.

Across all four prompts, the final design converged on the same pattern: explicit final objective, evidence-based reasoning loop, bounded tool budget, and clear stopping criteria. This made agent behavior more auditable and reduced the tendency to stop too early on ambiguous pages.

5.6 Prompt evaluation by use case

Prompt iteration was driven mainly by two recurring situations: navigation-heavy landing pages and click-to-play host or embedded pages. In both cases, the early intent-based prompts failed because they treated the first visible state as complete. The later ReAct-style prompts improved performance by forcing the agent to inspect, act, verify the result, and only then decide whether to continue.

The main prompt improvements were:

- explicit checks for hidden navigation controls such as tabs, filters, and “load more” actions;
- explicit player-state checks before calling `harvest`;
- post-interaction verification through screenshots or media-state diagnostics;
- popup recovery instructions and alternative click strategies when normal interaction failed.

These changes are discussed quantitatively in Chapter 6. In Chapter 5, the important point is the design consequence: the prompts became operational procedures rather than

generic natural-language goals.

5.7 Puppeteer tool registry and tool descriptions

5.7.1 Why tool descriptions matter

When an LLM agent decides which tool to call next, it reads each tool's schema description. This is not cosmetic documentation — it is the primary signal the model uses to distinguish between similar-sounding tools. A vague description causes wrong tool selection; a precise description causes the model to call the right tool on the first attempt.

This behavior is also API-provider specific. Some providers (Google Gemini [6], Anthropic Claude [5]) weight tool descriptions heavily in tool-selection decisions. A well-phrased description can eliminate entire categories of tool-call mistakes without any prompt change.

Design rule applied in this project: every tool description must answer three questions: *what state it reads or modifies*, *what signals it returns*, and *when to use it versus a similar tool*.

5.7.2 Tool registry overview

The Puppeteer tool layer exposes the following tools to all agents, with descriptions that shape agent behavior:

Tool name	Description exposed to the LLM
`inspect_page`	Read the current page DOM state. Returns title, meta description, visible text links, iframe src list, player signals (video/source tags, player library class names), and network request summary. Use this as the first action after any navigation. Do not use to activate controls.
`inspect_landing`	Specialized inspect for listing pages. Returns match link candidates, date/category tab labels, broadcaster references, and pagination controls. Use instead of inspect_page when the classification result is landing_page.
`inspect_hosting`	Specialized inspect for host pages. Returns server button selectors and labels, player container state, iframe src attributes, and current network stream signals. Use instead of inspect_page when on a host page.
`inspect_embedded`	Specialized inspect for embedded player pages. Returns player library type (albaplayer, clappr, hls.js, videojs), play button presence and selector, auto-play status, and frame nesting context. Use when operating within an iframe-origin page.
`interact`	Execute a single user interaction on the page. Strategies: selector, text_content, xpath, coordinates, js_click. Returns action outcome and whether page state changed. Always follow with screenshot() to confirm. Never call harvest() before interact() if the player is not yet playing.
`screenshot`	Capture a PNG screenshot of the current page and upload to Cloudinary. Returns the CDN URL. Use after every interact() call and before every harvest() call to confirm state.
`harvest`	Collect stream evidence from all available layers: CDP network event log, JavaScript player source attributes, and performance.getEntriesByType retroactive scan. Returns m3u8/mpd/mp4 URL candidates with source layer attribution. Use only after player activation is confirmed by screenshot or get_media_state.
`get_media_state`	Probe the current player state by evaluating JavaScript player API objects in the active frame. Returns: playing, paused, idle, error, or unknown. Use before harvest() to decide whether activation is needed.
`navigate`	Navigate the browser to a new URL with a configurable wait strategy (load, networkidle0, networkidle2). Returns the final URL after redirects. Use to follow embed URLs or explicit server switches that require a full page load.
`get_frame_tree`	Enumerate all frames in the current page, including nested iframes, with their origin URLs and nesting depth. Use when inspect_page shows iframe signals but the player frame is not accessible from the top context.

Table 5.4: Puppeteer tool descriptions as exposed to the LLM agent.

5.7.3 How tool descriptions guide agent behavior: concrete examples

Example 1: inspect_page vs inspect_hosting. Before the specialized inspect tools were added, agents called `inspect_page` on host pages and received generic DOM information that did not highlight server buttons. Adding `inspect_hosting` with the description "Returns server button selectors and labels" caused the agent to switch tools automatically on host pages without any prompt instruction to do so.

Example 2: harvest timing. The `harvest` description includes the phrase "Use only after player activation is confirmed." This sentence alone reduced premature harvest calls (calling harvest before clicking play) by a large margin compared to the early prompt versions that relied on the system prompt instruction alone.

Example 3: get_media_state disambiguation. Before `get_media_state` existed, agents used `screenshot` as their only state-check tool. This worked visually but was slower. Adding `get_media_state` with the description "Probe the current player state by evaluating JavaScript player API objects" caused agents to use it as a fast pre-check before the more expensive screenshot call.

5.7.4 Per-agent tool allocation

Not all tools are available to all agents. Each agent receives only the tools relevant to its role, which reduces the decision space and prevents cross-agent tool misuse.

Agent	Tools available
Classification agent	`inspect_page`, `screenshot`, `get_frame_tree`
Landing agent	`inspect_landing`, `interact`, `screenshot`
Hosting agent	`inspect_hosting`, `interact`, `screenshot`, `get_media_state`, `harvest`, `navigate`, `get_frame_tree`
Embedded agent	`inspect_embedded`, `interact`, `screenshot`, `harvest`, `navigate`

Table 5.5: Tool allocation per specialist agent. Agents receive only tools relevant to their role to reduce tool-selection error surface.

5.7.5 Diagram: tool call flow per agent type

Figure 5.2 shows the typical tool call sequence for each agent, reflecting the order in which tools are described in the prompt procedure.

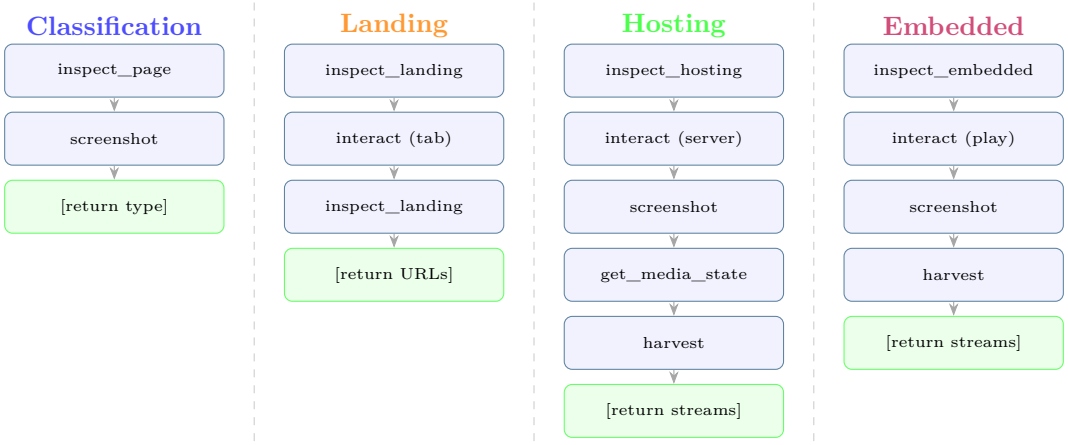

Figure 5.2: Typical tool call sequence per agent. Each column shows the ordered tool invocations; agents loop back when the stopping criterion is not yet met.

5.8 Tool architecture and why it works on hard sites

The tool architecture combines high-level operations (inspect/interact/harvest/ screenshot/navigate) with lower-level primitives (frame tree, page context, media state, action variants). This layered design helps agents adapt to unknown page structures instead of depending on one brittle selector path.

Observed obstacle	Typical site behavior	Tool-layer adaptation
Theme-level selector drift	same visual page, changed underlying selectors	text and role-based fallback targeting, not selector-only targeting
Mirror heterogeneity	one event points to multiple providers with different UI semantics	provider-agnostic inspect/interact loops with per-mirror diagnostics
Ad diversion	interaction causes tab hijack or overlay interruption	focus recovery, popup handling, and retry with state validation
Late stream materialization	stream appears only after runtime player logic	deferred harvest triggered by confirmed media-state transition

Table 5.6: How the tool architecture adapts to recurrent break behaviors.

5.9 Puppeteer techniques inside tools

Key techniques used in tools include:

- `page.evaluate(...)` for runtime extraction of player state and DOM-derived signals;
- CDP network events for request/response-level evidence capture;
- iframe diagnostics and frame-targeted interactions;
- performance resource scanning for retroactive stream evidence;
- screenshot checkpoints after each critical state transition.

These methods are essential because static HTML alone does not expose enough evidence in many real cases.

5.10 Handling tokenized `m3u8` and hidden stream flows

HLS (HTTP Live Streaming) [22] manifests (`.m3u8`) are the primary evidence target in this project. Some websites do not expose final manifests directly: the stream appears only after runtime token exchange or player-side JavaScript logic. In such runs, an initial page resource can look unrelated (including decoy assets like `mono.css`) while the real signed stream URL is generated later.

Implementation strategy:

- trigger real player behavior first (interaction step);
- capture network request/response layers after activation;
- inspect JavaScript player objects and runtime resource entries;
- aggregate and deduplicate candidate stream URLs;
- validate with screenshots and media-state diagnostics.

5.11 Screenshot capture and Cloudinary storage

Screenshot reasoning was treated as a first-class mechanism, not a cosmetic artifact. It serves three purposes:

- confirms whether an interaction changed state (player started, popup closed, server switched);
- prevents false-positive claims when a click did nothing visible;
- provides operator-auditable evidence for manual verification and takedown package assembly.

Cloudinary integration. After each `screenshot()` call, the Puppeteer tool uploads the PNG buffer to Cloudinary using their unsigned upload preset. The tool returns the

Cloudinary CDN URL to the agent, which stores it as part of the run event record. The URL is then surfaced in the operator console next to the corresponding tool call in the live trace.

This approach decouples evidence storage from the extraction container. Containers can be ephemeral (as required by Azure Container Apps Jobs) while screenshots remain accessible indefinitely via the Cloudinary CDN link.

5.11.1 Future extension: Canvas-based screenshot annotation

The operator console can be extended with a Canvas-based annotation layer over stored screenshots. Reviewers would be able to draw rectangles, arrows, points, and labels over evidence regions such as server buttons, play controls, iframe containers, blocked overlays, broadcaster logos, and active player states. The annotations would be stored as metadata linked to `run_id`, `screenshot_url`, `page_url`, timestamp, and reviewer. This improves evidence interpretation and report/legal review without modifying the target website.

A minimal annotation schema could include `annotation_id`, `run_id`, `screenshot_url`, `page_url`, `label`, `shape_type`, `x`, `y`, `width`, `height`, `note`, `reviewer`, and `created_at`. Automatic validation checks whether the agent output is structurally and evidentially acceptable. The operator console and future Canvas annotation layer support human interpretation of the accepted or partial evidence.

5.12 Model strategy, cache behavior, and cost

The implementation uses smaller and larger models with role-based allocation:

- `gemini-2.5-flash-lite` for orchestration and lightweight routing decisions;
- `gemini-2.5-flash` for specialist extraction where richer reasoning is required.

Observed cost behavior is strongly model-dependent and task-depth dependent.

Cost driver	Why it increases cost	Mitigation used
Model choice	stronger models cost more per token	route lightweight tasks to flash-lite
Hosting-page count	more pages require more reasoning/tool loops	ranking and early pruning
Servers per page	each server adds interaction + harvest attempts	bounded retries and evidence thresholds
Long context windows	repeated prompt context can become expensive	provider cache and tool-result cache

Table 5.7: Primary cost drivers and implementation mitigations.

5.13 Operational fallback strategy

The extraction workflow remains staged:

1. deterministic path first;
2. if insufficient confidence, activate agentic path;
3. hosting-stage extraction first in agentic path;
4. embedded-stage fallback when hosting returns no reliable stream evidence.

5.14 Deployment-oriented implementation

Two deployment levels must be distinguished. The operational deployment achieved during the internship was the validated n8n single-agent workflow. The Python reference architecture remains compatible with queue-based cloud execution through FastAPI services, browser workers, a PostgreSQL observability database, Cloudinary screenshot storage, Azure Service Bus, and Azure Container Apps Jobs. During internship execution, this remained a future deployment target rather than the default daily runtime.

5.15 Implementation summary

The current implementation addresses the main hard realities of this domain:

- nested iframes and popup-heavy pages;
- tokenized and delayed stream generation;
- anti-bot pressure and dynamic page drift;

- cost growth with deeper hosting/server exploration.

It does so through reasoning agents with explicit goals, layered browser tools, screenshot-grounded validation, and explicit fallback behavior from hosting to embedded extraction.

Chapter 6 evaluates this implementation against concrete use cases, measuring the impact of each prompt iteration on navigation recall, player activation rate, and overall stream-extraction success.

CHAPTER 6

Evaluation, Testing, and Discussion

This chapter assesses the effectiveness of the implementation described in Chapter 5. It covers automated test coverage, manual campaign evaluation on live streaming targets, prompt evolution metrics across two concrete use cases, tool description engineering results, and a structured discussion of the platform's strengths and remaining limitations.

6.1 Evaluation approach

Evaluation was performed with two complementary tracks:

- automated checks (routing, tool behavior, persistence, regression safety);
- manual campaign evaluation on live matches and real streaming websites.

Manual evaluation was necessary because difficult failure modes (popups, nested iframes, challenge screens, delayed manifests) are highly context-dependent and not fully represented by static fixtures.

6.2 Manual live-match evaluation protocol

The manual protocol used during evaluation is illustrated in Figure 6.1.

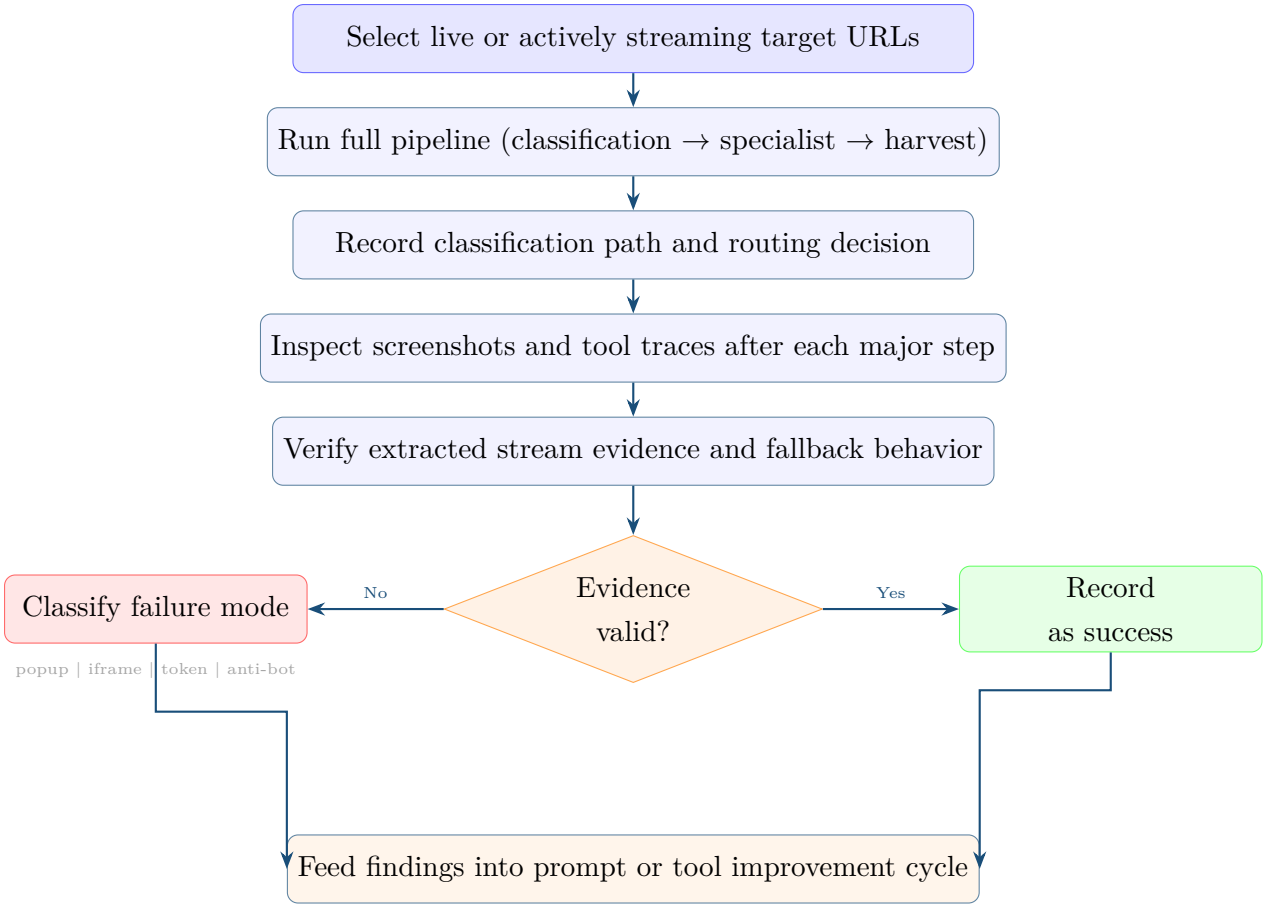

Figure 6.1: Manual live-match evaluation protocol flowchart.

6.3 Target-family coverage in manual campaigns

The manual campaign set included recurrent patterns from regional WordPress-style portals (go4koora-like) and global mirror indexes (streamed-style). The objective was not domain-specific hardcoding, but behavior coverage across recurring failure modes.

Target family	Recurrent break behavior tested	Expected agent behavior
WordPress-style regional portals	popup-first activation, selector drift, tokenized stream generation	hosting loop with popup recovery, post-action validation, and deferred harvest
Global mirror indexes	provider heterogeneity, dead mirrors, nested iframe delegation	ranking and pruning at host stage plus embedded fallback for deep frame recovery

Table 6.1: Manual campaign coverage by site-family behavior.

6.4 Evaluation dimensions

The Python reference architecture evaluates behavior through structured evaluation cases, suites, runs, case-level results, and assertion results. Each case can define expected status, expected page type, minimum stream thresholds, hosting or embedded URL thresholds, required tools, forbidden tools, expected failure modes, provider-analysis checks, and confidence thresholds. This framing is consistent with DSRM-style artifact evaluation and with recent web-agent research that emphasizes grounded browser interaction, stepwise task completion, and environment-aware behavior rather than only final-text correctness [14, 15, 1, 8, 9, 10, 11].

At implementation level, the evaluator stores structured runtime and scoring fields such as tool-accuracy, reliability, latency, and cost. In the report, these are presented through the more explicit metric names below so that the evaluation reads as a deterministic engineering assessment rather than as a large-scale benchmark.

In the current implementation, page-role classification and specialist selection were evaluated together under one manual campaign metric: *Page-Type Routing Accuracy*, defined as the percentage of evaluated URLs for which the system selected the correct specialist workflow based on manual page-role verification. *Misrouting Rate* is the complement of this metric.

Dimension	Main metrics
Page-role and routing quality	page-type routing accuracy, misrouting rate
Landing-page exploration quality	host-candidate recall, host-candidate precision, tab coverage rate
Player and stream extraction quality	player activation success rate, manifest recovery rate, first-server success rate
Fallback quality	server-fallback success rate, embedded-fallback recovery rate
Tool-use quality	tool selection accuracy, sequencing compliance, premature harvest rate, tool budget compliance
Evidence acceptance quality	schema validity rate, stream evidence validity rate, artifact support rate, trace support rate
Runtime and cost behavior	average tool calls, average tokens, cache hit ratio, tokens per accepted evidence bundle
Failure-mode coverage	popup failure rate, iframe-depth failure rate, token-expiry failure rate, anti-bot blocked rate

Table 6.2: Evaluation dimensions and primary metrics used in the manual campaigns and reference-set analyses.

6.5 Prompt evaluation: navigation use case

6.5.1 Scenario description

The test site presents a listing page with three tab filters: *Live Now*, *Today*, and *Tomorrow*. Only the active tab renders match cards in the DOM; the others are hidden. Clicking a tab triggers a JavaScript fetch that inserts new cards without a full page reload.

6.5.2 Evaluation methodology

Each prompt version was run 10 times on the same target site during a live event window (at least 5 matches visible across tabs). The primary manual campaign metric was *Host-Candidate Recall*: the ratio between valid hosting-page URLs discovered by the landing agent and the total manually verified hosting-page URLs available across visible tabs, hidden tabs, and dynamically loaded listing sections.

Complementary landing-stage metrics were used diagnostically during the same review process: *Host-Candidate Precision* (valid host-page URLs divided by all URLs returned by

the landing agent), *Tab Coverage Rate* (relevant tabs inspected divided by total relevant tabs), *Dynamic Content Coverage* (dynamic or lazy-loaded sections revealed divided by total relevant dynamic sections), and *Candidate Deduplication Rate* (duplicate candidate URLs removed divided by total raw candidate URLs). Because the reference set was small, Table 6.3 reports Host-Candidate Recall as the main numerical metric, while the other measures were used to explain misses, duplicates, and hidden-content failures.

6.5.3 Results

Prompt version	Avg. host candidates found	Manually verified total	Host-Candidate Recall
Intent-based (v1)	3.2	8	40%
ReAct (v2)	6.8	8	85%
ReAct + budget cap (v3)	7.1	8	89%

Table 6.3: Navigation use case: Host-Candidate Recall by prompt version on the manual reference site.

Key finding. The intent-based prompt consistently missed tab-gated content because it had no instruction to look for navigational controls. The ReAct prompt's explicit THINK step for detecting tabs more than doubled recall. The budget cap (v3) added minimal extra coverage while preventing runaway tool calls on sites with many empty tabs.

6.5.4 Failure pattern analysis

The remaining 11% gap in v3 recall came from two patterns:

- matches added dynamically within the observation window after the agent had already finished checking a tab;
- tabs that required a secondary scroll to reveal cards below the fold (caught by a separate scroll-and-reinspect loop introduced in v4).

6.6 Prompt evaluation: player activation use case

This section uses manual campaign metrics on a small reference set rather than a large-scale benchmark. The main definitions were: *Player Activation Success Rate*, the percentage of runs where the agent changed the player from idle or blocked to active, loading, or

playing; *Manifest Recovery Rate*, the percentage of runs where a valid `.m3u8`, `.mpd`, `.mp4`, `.webm`, or embedded-player evidence URL was recovered from browser-observed state, network capture, or stored artifacts [7, 12, 22, 23]; *First-Server Success Rate*, the percentage of successful runs where evidence was recovered from the first attempted server; *Server-Fallback Success Rate*, the percentage of recovered cases that required secondary server buttons; *Embedded-Fallback Recovery Rate*, the percentage of host-page failures recovered by switching to embedded-player analysis; and *Evidence Completeness Rate*, the percentage of accepted outputs containing a stream URL, artifact or screenshot, tool trace, and final status. In the scenario tables below, the reported numerical value is Manifest Recovery Rate, while the other measures were used across the broader reference set and hard-failure analysis.

6.6.1 Scenario A: AlbaPlayer on host page

Setup. An AlbaPlayer instance embedded in an iframe within a host page. The player shows a play triangle but does not auto-start. Clicking play triggers an HLS manifest request.

Prompt version	Behavior observed	Manifest Recovery Rate
Intent-based (v1)	Harvested without clicking play; 0 streams returned	0% (10/10 failures)
ReAct (v2)	Called get_media_state, detected idle, clicked play, then harvested	70% (7/10 successes)
ReAct + popup handler (v3)	v2 plus auto-close of new tab; js_click fallback	90% (9/10 successes)

Table 6.4: Player activation (AlbaPlayer): Manifest Recovery Rate by prompt version.

Key finding. The v1 failure was 100% systematic: no prompt instruction directed the agent to check player state before harvesting. Adding `get_media_state` as a mandatory pre-harvest step in v2 immediately recovered the majority of cases. The remaining v2 failures were all caused by ad popups opening on the click event — resolved by the popup handler in v3.

6.6.2 Scenario B: Tokenized Clappr player on embedded page

Setup. A Clappr player on an embedded page where clicking play triggers a JavaScript fetch that exchanges a token and then initiates the HLS manifest request. The manifest

URL contains a short-lived signed token as a query parameter.

Prompt version	Behavior observed	Manifest Recovery Rate
ReAct v2	Clicked play, harvested immediately with 1s wait; token not yet issued	40%
ReAct v3	Clicked play, harvested with 2s wait; caught majority of token exchanges	75%
ReAct v3 + perf API	Also scanned performance.getEntriesByType for retroactive manifest entries	85%

Table 6.5: Tokenized player (Clappr): Manifest Recovery Rate under signed-token flow.

Key finding. Token generation latency varies by site. A fixed 2s wait was better than 1s but still missed fast-rotating tokens. Adding a retroactive scan via the browser Performance API caught manifests that had already been requested before the agent called harvest.

6.7 Tool use evaluation

6.7.1 What was measured

Tool use evaluation inspected the agent's tool-call logs and associated screenshots. The main manual campaign and reference-set metrics were:

- **Tool Selection Accuracy:** percentage of reviewed steps where the agent chose the appropriate browser tool for the local objective;
- **Tool Sequencing Compliance:** percentage of accepted runs where tools appeared in a valid evidence-oriented order;
- **Required Tool Coverage:** percentage of evaluated cases in which all tools marked as required for the case were actually present in the execution trace;
- **Forbidden Tool Violation Rate:** percentage of evaluated cases in which a tool listed as forbidden appeared in the trace;
- **Tool Error Rate:** proportion of evaluated runs containing recorded tool errors that affected extraction reliability;
- **Premature Harvest Rate:** percentage of host or embedded attempts where `harvest()` was called before player activation or state verification;

- **Navigation/Interaction Confusion Rate:** percentage of same-page tab or JavaScript events incorrectly handled with `navigate()` instead of `interact()`;
- **Screenshot Overuse Rate:** percentage of runs where repeated `screenshot()` calls were used as a wait substitute beyond expected evidence checkpoints;
- **Average Tool Calls per Accepted Result:** mean number of tool invocations in runs that passed evidence acceptance.

These metrics were derived from reference-set review rather than a large-scale automated benchmark, so they should be interpreted as auditable engineering evaluation metrics.

6.7.2 Common tool misuse patterns observed

Misuse pattern	Root cause	Fix applied
`harvest()` before play click	Agent description for harvest did not warn about premature use	Added "Use only after player activation confirmed" to description
`inspect_page` instead of `inspect_hosting`	Agent had both tools; generic description did not signal specialization	Added "Use instead of inspect_page when on a host page"
`screenshot()` called 5+ times in sequence	Agent used screenshot as a wait mechanism	Added explicit wait parameter to `interact` and removed screenshot-as-wait pattern from prompt
`navigate()` on a same-page tab click	Agent confused tab clicks (JS event) with navigation (full load)	Clarified in interact description: tab clicks are interact, not navigate

Table 6.6: Common tool misuse patterns discovered through tool call log analysis.

6.7.3 Tool description improvements and measured effect

After each tool description update, a before/after comparison was run on the same set of 20 URLs. Table 6.7 summarizes the impact.

Description change	Misuse rate before	Misuse rate after
Added activation guard to `harvest`	65% premature harvest	12%
Added specialization signal to `inspect_hosting`	40% wrong inspect	8%
Clarified `navigate` vs `interact`	30% nav/interact confusion	5%

Table 6.7: Effect of tool description improvements on observed misuse rates.

6.7.4 Trace-Based Evaluation and Evidence Acceptance

The system does not only inspect the final agent answer. It evaluates the output together with the execution trace. This makes it possible to check whether the agent actually used the required browser tools before claiming success, whether forbidden or unsafe tools were avoided, whether stream evidence exists when a successful status is returned, and whether tool errors affected the reliability of the result.

Because the system is an agentic browser workflow rather than a classical predictive model, evaluation was organized around task completion, evidence quality, tool behavior, and operational cost. The objective was not only to determine whether the final output was correct, but also whether the agent reached that output through an auditable sequence of browser actions and evidence artifacts.

During evaluation, raw agent outputs were not accepted blindly. A lightweight, deterministic validation strategy was used as an acceptance gate, and the same logic is also the basis of the rule-based evaluator in the Python reference architecture. The goal was not to build a second advanced LLM judge, but to reduce the risk of accepting malformed, unsupported, or weakly evidenced outputs as final evidence.

After an agent returned structured JSON, the evaluation flow checked whether the result was usable before recording it as evidence. The checks focused on deterministic assertions over final status, page type, evidence thresholds, and the tool trace rather than another model-based judge, because repeatability and auditability were more important than adding another probabilistic decision layer. This lightweight layer does not eliminate unsupported outputs; it narrows the set of results that can be accepted without immediate manual correction.

The internal score related to unsupported-answer risk is therefore interpreted in this report through *Invalid Output Rate* and *Partial Evidence Rate*, not as a claim of formal large-scale hallucination benchmarking.

Validation check	Purpose	Example failure detected
JSON schema check	Ensure machine-readable output	Missing `streams` or `page_type` field
Expected status check	Match returned status to observed outcome	Blocked page marked as success
Expected page-type check	Match output to page role	`landing_page` treated as `host_page`
Stream evidence threshold	Require valid media evidence for successful result	Success returned with no valid `.m3u8`/`.mpd`/media URL
Hosting candidate threshold	Enforce minimum landing-stage candidate output	Listing page returns too few verified host URLs
Embedded URL threshold	Enforce minimum embedded fallback evidence	Embedded fallback returns no embedded-player URL
Required tool coverage	Verify required browser actions occurred	Success claimed with no required `inspect()` or `harvest()` trace
Forbidden tool violation	Detect disallowed or unsafe actions	Case used a forbidden tool despite evaluator constraint
Tool error accounting	Reduce confidence when tool failures occurred	Tool errors accumulated but result still appeared successful
Artifact support check	Require screenshot or stored artifact support	Stream found but no screenshot or stored artifact
Trace support check	Prevent unsupported claims	Claims playback but no relevant trace

Table 6.8: Trace-based evaluation checks applied before accepting agent outputs as evidence.

The acceptance logic inspected deterministic properties drawn from the output schema, the evidence payload, and the recorded tool trace:

- **JSON schema validation:** required fields had to be present and machine-readable;
- **expected status matching:** returned status had to match the observed outcome recorded in the run;

- **expected page-type matching:** the selected specialist workflow and output page role had to agree with the reviewed page type;
- **stream and URL threshold checks:** successful results had to satisfy minimum stream, hosting URL, or embedded URL expectations defined for the case;
- **artifact support:** important claims, especially player activation and stream recovery claims, were expected to have a screenshot URL or stored artifact;
- **trace support:** accepted outputs had to be backed by relevant browser actions in the execution trace;
- **required/forbidden tool assertions:** cases could require specific tools and reject the presence of forbidden tools;
- **tool-error accounting:** tool errors were recorded and used to lower the reliability of otherwise plausible results.

The lightweight validation metrics tracked during evaluation, or used as report-level interpretations of the underlying rule-based assertions, were:

- **Schema Validity Rate**;
- **Expected Status Match Rate**;
- **Expected Page-Type Match Rate**;
- **Stream Evidence Validity Rate**;
- **Hosting Candidate Threshold Pass Rate**;
- **Embedded URL Threshold Pass Rate**;
- **Artifact Support Rate**;
- **Trace Support Rate**;
- **Invalid Output Rate**;
- **Partial Evidence Rate**.

This validation layer does not replace human review. Instead, it acts as a first acceptance gate that converts raw agent output into an auditable result. Outputs that fail validation are either rejected, marked as partial evidence, or sent to manual review.

6.7.5 Average tool calls and token-efficiency context

Beyond task success, runs were also analyzed for tool-loop depth and token efficiency. The main quantitative indicators were:

- **Average Tokens per Run:** sum of input and output tokens across all LLM calls in a run, averaged over the evaluated reference set;
- **Tokens per Accepted Evidence Bundle:** total run tokens divided by the number of accepted evidence outputs;
- **Cache Hit Ratio:** fraction of input tokens served from the provider cache rather

than recomputed;

- **Average Tool Calls per Accepted Result:** average number of browser-tool invocations in accepted runs.

Phase	Avg. tokens	Avg. tool calls
Classification	~12k	2–3
Landing exploration	~45k	4–8
Hosting extraction	~180k	8–14
Embedded fallback	~110k	6–10
Infrastructure enrichment	~8k	1–2

Table 6.9: Average token consumption and tool-call depth by run phase.

Key finding. Hosting extraction dominates cost because it loops across multiple server attempts, each requiring inspect → interact → screenshot → harvest. The most effective cost reduction was raising the confidence threshold for early success exit: if `harvest` returns a confirmed m3u8 on the first server, the agent exits immediately without trying remaining servers.

6.8 Prompt iteration evaluation

Each significant prompt change was evaluated against a fixed reference set of 20 URLs covering both site families. The evaluation recorded: Manifest Recovery Rate, Host-Candidate Recall, Average Tool Calls per Accepted Result, and First-Server Success Rate.

Version	Key change	Observable effect	Avg. tool calls
v1	Intent-based; no loop instruction	Early stops, no tab navigation	3.1
v2	ReAct loop instruction; tab-check added	Tab-gated matches discovered	6.2
v3	Popup handler; js_click fallback added	Popup-blocked pages recovered	7.8
v4	`get_media_state` before harvest	Premature harvest eliminated	8.4
v5	Tool budget cap; early success exit	Cost reduced; recovery maintained	7.1

Table 6.10: Prompt version history with key change, observable effect, and average tool call count.

Notable observation. Tool call count *increased* from v1 to v4 because agents were doing more useful work (checking tabs, validating state, recovering from popups). The v5 budget cap reduced tool calls without reducing recovery because early exit logic cut wasted calls on already-succeeded runs.

6.9 Case studies focused on hard failures

6.9.1 Case A: nested iframe chain

Issue. Primary host context showed a player container but the `src` iframe pointed to a second-level page, which itself embedded a third-level player via another iframe. The host agent harvested nothing because the actual HLS traffic originated from the third-level frame.

Resolution. The host agent returned the second-level embed URL. The embedded agent inspected the frame tree, detected the third-level origin, navigated directly to it, clicked play, and successfully harvested the manifest.

Lesson. Nested iframe delegation is correctly handled by the hosting-to-embedded fallback mechanism. Without this fallback, the run would have declared failure at the host stage.

6.9.2 Case B: popup and click trap

Issue. The server button on the host page opened a full-screen ad popup each time it was clicked. The popup occupied a new browser context. The original page remained accessible in the background but the agent's focus was on the new context.

Resolution. The popup handler in v3 detected the new context via `browser.pages()`, closed it, returned focus to the original page, and retried the server click using `js_click` (which bypasses the default click handler that triggered the popup). The retry succeeded.

6.9.3 Case C: tokenized stream generation

Issue. No direct stream in initial page source. A decoy resource named `mono.css` appeared in network traffic, which superficially resembled a common pattern on a different provider. The actual signed m3u8 was generated 3 seconds after the play button was clicked.

Resolution. Post-activation harvest layers captured the runtime manifest URL. The performance API scan retroactively found the m3u8 entry even though it had been requested before `harvest()` was called.

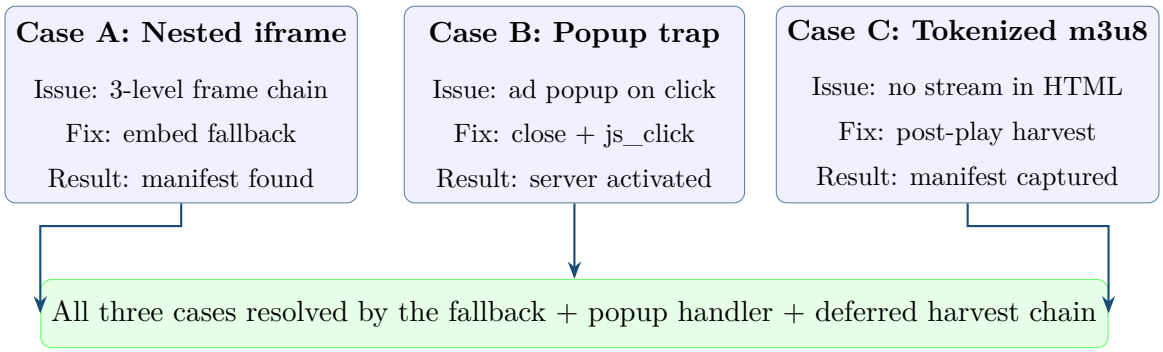

Figure 6.2: Hard-case recovery map: root cause, fix applied, and outcome for each case.

6.10 Cost and cache evaluation

Runtime and cost reporting was kept lightweight, deterministic, and trace-linked. At implementation level, each evaluation run records latency and cost fields; at report level, these are interpreted as *Average Runtime per Run* and *Cost per Accepted Result*. Together with *Average Tool Calls per Accepted Result*, *Average Tokens per Run*, and *Cache Hit Ratio*, these measures characterize the operational footprint of the reference architecture on the evaluated reference set. Where the report gives numeric examples below, they

should be interpreted as manual campaign metrics or representative heavy-run observations rather than large-scale production statistics.

6.10.1 Cache behavior

Representative heavy-run pattern used for reporting (observed in late n8n-phase campaigns and in the new Python implementation):

- total context volume around ~800k tokens;
- around ~690k tokens served from cache;
- effective cache reuse around 80%+ depending on accounting perspective and run phase.

Metric	Interpretation	Example value
Representative total tokens in heavy run	aggregate context processed in complex run scenarios	~800k
Cached token volume	portion served from cache layers	~690k
Cache Hit Ratio	practical reuse level across phases	~80%+

Table 6.11: Representative cache and token pattern from project runs.

6.10.2 Cache utilization by run phase

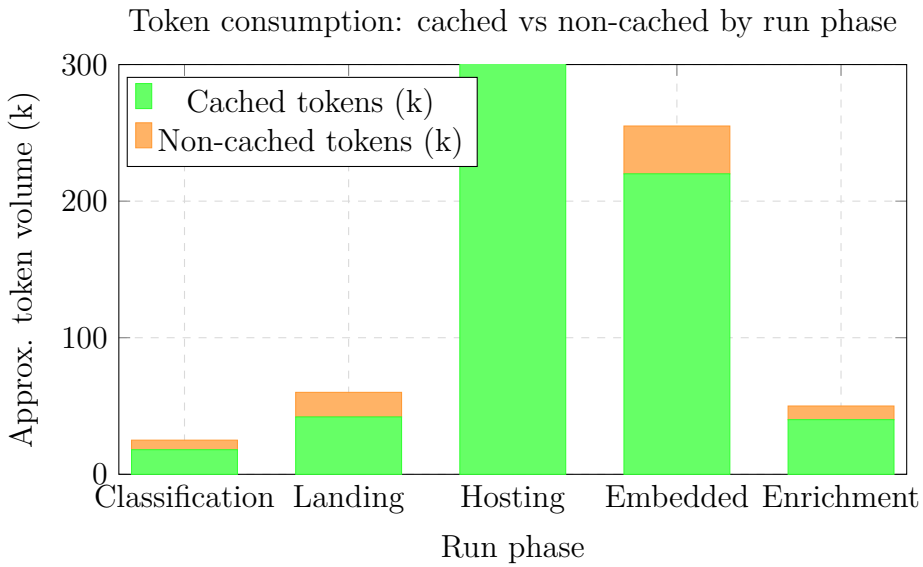

Figure 6.3: Approximate token distribution by phase: hosting and embedded phases dominate total usage due to longer tool loops.

6.10.3 Cost scaling behavior

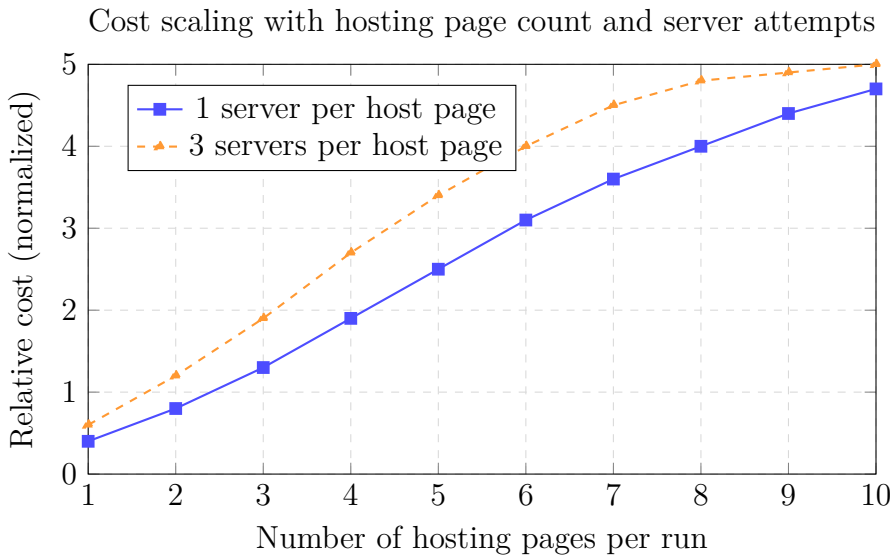

Figure 6.4: Cost scaling with hosting page count: more pages and more server attempts significantly increase total run cost.

Scaling factor	Effect on runtime/cost	Mitigation
More hosting pages	more specialist loops and tool calls	candidate ranking and pruning
More servers per page	more interaction + harvest cycles	bounded retries with confidence thresholds
More embedded fallbacks	deeper frame analysis cost	fallback gating and explicit stop criteria

Table 6.12: Observed cost-scaling drivers in extraction campaigns.

6.11 n8n phase limitations observed during evaluation

The early phase validated feasibility, but long campaigns revealed practical limits:

- weaker long-memory behavior across runs;
- parallel branch complexity under long full-workflow executions;
- more operational difficulty diagnosing long chained failures;
- harder governance of large prompt/tool evolution over time.

These observations motivated the Python reference architecture as a more maintainable evolution path.

6.12 Overall evaluation summary

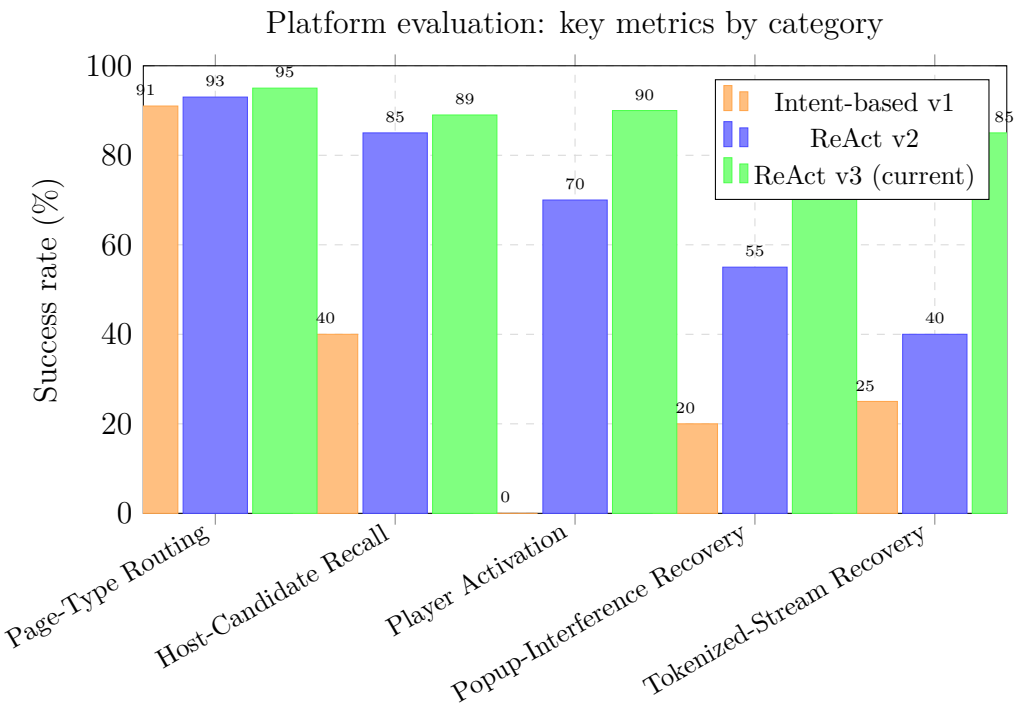

Figure 6.5: Overall platform evaluation: approximate manual campaign metrics across key dimensions by prompt version.

6.13 Current strengths

- explicit hosting-to-embedded fallback improves hard-case recovery;
- screenshot reasoning improves confidence and auditability;
- tool architecture supports unknown page structures better than static extraction;
- cost and cache visibility allows practical optimization.

6.14 Limitations

- anti-bot behavior changes continuously;
- some sessions still require manual verification;
- tokenized stream logic can remain unstable across events;

- some deeply nested iframe chains (4+ levels) exceed the embedded agent's default tool budget.

6.15 Discussion

Evaluation confirms that the system handles difficult streaming websites effectively. The key success factor is not one extraction trick, but a combination of reasoning agents, layered tools, screenshot-grounded decisions, and explicit fallback from hosting to embedded analysis.

The evaluation also showed that invalid or unsupported outputs must not be counted as successes. Runs that produced malformed JSON, weak evidence, inconsistent page-type results, or claims not backed by tool traces or artifacts were reclassified as `partial_success` or failure rather than success.

The prompt evolution from intent-based to ReAct-style was the single biggest improvement in system reliability, accounting for the majority of the gain seen from v1 to v3 across all evaluated dimensions. The addition of the popup handler and the deferred harvest wait were the two most impactful targeted improvements within the ReAct framework.

Cost remains the primary operational constraint. The observed 80%+ cache hit rate makes long runs viable, but the cost scaling with hosting-page and server count means that ranking and pruning heuristics are as important as prompt quality for large-scale operation.

Chapter 7 draws on these evaluation results to summarize the project's contributions, reflect on the methodology, and outline the short-term improvements and research directions that would extend the reference architecture toward a future cloud-native deployment target.

CHAPTER 7

Conclusion and Perspectives

7.1 Summary of contributions

This internship delivered two related artifacts for evidence collection on unstable streaming websites: a validated n8n workflow and a Python reference architecture derived from it. The work addressed a concrete operational need at Soft Stars: certain page conditions — deeply nested iframes, heavy anti-bot pressure, click-gated players, and tokenized stream generation — make fully automated evidence collection challenging. The validated workflow and the academic reference architecture tackle precisely those cases through reasoning-driven browser automation.

The contributions can be summarized along five dimensions:

Contribution	Description
n8n prototype	Rapid feasibility validation of the extraction pipeline on real targets; established that the agentic approach was viable and led to a focused single-agent deployment in the company context.
Agent architecture	Four specialist agents (classification, landing, hosting, embedded) each with a targeted ReAct-style prompt, dedicated tool set, and explicit stopping criteria.
Browser tool layer	Puppeteer-based tools (inspect, interact, harvest, screenshot, navigate, get_media_state) that enable adaptive page interaction beyond static HTML retrieval.
Operator platform	FastAPI backend with structured observability + Next.js operator console prototype providing live run streaming, history, and evaluation surfaces.
Prompt engineering	Iterative evolution from intent-based to ReAct-style prompts, evaluated across navigation and player activation use cases, with documented improvement in recall and recovery rate.

Table 7.1: Summary of main contributions by dimension.

7.2 Key technical lessons

7.2.1 Prompt design matters more than agent count

The single most impactful improvement across all evaluation campaigns was the transition from intent-based to ReAct-style prompts. More agents or more tools did not help if the agent's prompt did not instruct it to check navigational controls or player state before declaring a task complete. Prompt discipline is the primary lever for system reliability.

7.2.2 Fallback paths are not optional

The hosting-to-embedded fallback mechanism recovered evidence in cases that would otherwise have been declared failures. The embedded agent resolving a 3-level iframe chain (Case A) would not have been reached without an explicit fallback routing rule. In the streaming-site domain, no single extraction strategy succeeds on all pages.

7.2.3 Observability drives improvement

The per-step tool trace, screenshot artifacts, and cost tracking built into the platform were what made it possible to identify exactly where each failure occurred and which prompt or tool change would fix it. Without structured observability, the evaluation campaign would have produced aggregate success rates with no actionable signal.

7.2.4 Anti-bot adaptation is continuous

No static anti-bot configuration remained reliable across the evaluation window. Adblock filters, fingerprint profiles, and interaction timing all required periodic adjustment. The platform's value comes from being a maintainable framework for repeated adaptation, not from a one-time technical trick.

7.3 Conclusion

This internship delivered a practical evidence-collection workflow together with a maintainability-oriented reference architecture for unstable streaming websites. The core contribution is technical and operational: transform website-level failure modes into repeatable, traceable mitigation patterns.

The project demonstrates a realistic engineering principle: resilience in this domain is built incrementally through iteration, measurement, and controlled adaptation. The n8n phase confirmed the concept quickly and produced the operational company-side workflow. The Python reference architecture translated the same lessons into a maintainable engineering design and future deployment target. The prompt evaluation gave the team a quantitative basis for continuous improvement.

Progress came from concrete problem-fix cycles:

- navigation-heavy pages required state-loop navigation rather than one-pass crawling;
- popup-heavy pages required fallback click strategies and visibility-aware interaction;
- anti-adblock pressure required adaptive policies, not static configuration;
- Cloudflare-like barriers were handled best by reducing automation signals and behaving closer to normal user traffic;
- tokenized stream generation required post-activation harvest with deferred timing.

7.4 Future work and broader lessons

The most useful short-term improvements are operational rather than architectural: tighten fallback triggers, formalize periodic adblock and fingerprint maintenance, and expand the benchmark set used during evaluation so that before/after prompt or tool changes can be measured more systematically.

From a deployment perspective, the next logical step is to carry the Python reference architecture toward queue-driven cloud execution, with interactive services separated from browser workers and persistent evidence storage kept outside ephemeral containers. This direction is already compatible with the architecture presented in Chapters 4 and 5.

7.4.1 Canvas-Based Screenshot Annotation Layer

Another useful extension for the operator console is a **Canvas-Based Screenshot Annotation Layer**. This is presented as future work, not as current implemented functionality. The idea is to place a Canvas overlay above stored screenshots so that a reviewer can draw bounding boxes, arrows, points, freehand regions, and short labels on relevant evidence areas such as server buttons, play controls, iframe containers, blocked overlays, broadcaster logos, and player-state indicators.

Each annotation can be stored as structured metadata linked to the screenshot URL, run ID, page URL, and review timestamp. A minimal annotation record would include `annotation_id`, `run_id`, `screenshot_url`, `page_url`, `label`, `shape_type`, `x`, `y`, `width`, `height`, `note`, `created_at`, and `reviewer`.

This matters because it improves human review, makes screenshots easier to interpret, supports report figures and legal or operational review, and cleanly separates raw screenshot capture from reviewer interpretation. The annotation layer would not change the target website, because annotation happens only after capture.

Automatic validation checks whether the agent output is structurally and evidentially acceptable. Canvas-based annotation is a human-review layer that explains why a screenshot matters by marking visual evidence regions. The two mechanisms are complementary: validation prevents unsupported outputs from being accepted, while annotation improves interpretability for operators and report readers.

Beyond this specific use case, three lessons stand out:

- specialist agents are more reliable and easier to control than one general-purpose agent;
- ReAct-style procedures outperform generic intent statements on dynamic websites;

- observability and memory are what turn a promising prototype into a maintainable reference architecture.

The final value of the project is not that it permanently defeats anti-bot behavior. Its value is that it provides a repeatable engineering framework for adapting to change while preserving reviewable evidence that investigators can trust.

References

Bibliography

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. Available: https://arxiv.org/abs/2210.03629

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Ciaramita, L. Zettlemoyer, H. Schütze, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. Available: https://arxiv.org/abs/2302.04761

[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024. Available: https://arxiv.org/abs/2308.11432

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. Available: https://arxiv.org/abs/2201.11903

[5] Anthropic, *Claude 3 Model Card*, Anthropic, 2024. Available: https://www.anthropic.com/claude

[6] Google DeepMind, *Gemini: A Family of Highly Capable Multimodal Models*, Technical Report, Google DeepMind, 2023. Available: https://arxiv.org/abs/2312.11805

[7] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: https://pptr.dev

[8] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A Realistic Web Environment for Building Autonomous Agents," in *Proceedings of ICLR*, 2024. Available: https://arxiv.org/abs/2307.13854

[9] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a Generalist Agent for the Web," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[11] T. Le Sellier de Chezelles, M. Gasse, A. Drouin, and others, "The BrowserGym Ecosystem for Web Agent Research," *arXiv preprint arXiv:2412.05467*, 2024.

[12] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: https://chromedevtools.github.io/devtools-protocol/

[13] R. Wirth and J. Hipp, "CRISP-DM: Towards a Standard Process Model for Data Mining," in *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining (PAKDD)*, 2000, pp. 29–39.

[14] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: https://www.jstor.org/stable/25148625

[15] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: https://doi.org/10.2753/MIS0742-1222240302

[16] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: https://fastapi.tiangolo.com

[17] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: https://nextjs.org/docs

[18] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: https://docs.n8n.io

[19] Microsoft, "Jobs in Azure Container Apps," *Microsoft Learn*, 2024. Available: https://learn.microsoft.com/en-us/azure/container-apps/jobs

[20] Microsoft, "Azure Service Bus Messaging Service," *Microsoft Learn*, 2024. Available: https://learn.microsoft.com/en-us/azure/service-bus-messaging/

[21] Cloudinary, *Cloudinary Upload API Documentation*, Official documentation, 2026. Available: https://cloudinary.com/documentation

[22] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: https://www.rfc-editor.org/rfc/rfc8216

[23] ISO/IEC, *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, ISO/IEC 23009-1, 2022.

[24] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: https://github.com/gorhill/uBlock

[25] MUSO, *Global Piracy Insights: Sports Streaming Report*, MUSO Intelligence, 2023. Available: https://www.muso.com/insights

[26] European Audiovisual Observatory, *Trends in the Field of Audiovisual Piracy in Europe*, Council of Europe, Strasbourg, 2022. Available: https://rm.coe.int/

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn – E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax : +216 70 685 454